Adaptive Memory-Based Cuckoo Search for High-Dimensional Feature Optimization in Non-Stationary Fraud Detection Systems

Albert Ankomah Dodoo[1][0000-1111-2222-3333] ,Justice Kwame Appati, Daniel Kwame Amissah, Leonard Mensah Boante, Julius Yaw Ludu, Micheal Agbo Tettey Soli

[1] Department of Computer Science, University of Ghana, Accra, Ghana
aadodoo003@st.ug.edu.gh, jkappati@ug.edu.gh, damissah002@st.ug.edu.gh, lboante@st.ug.edu.gh, jyludu@ug.edu.gh, msoli@ug.edu.gh

Abstract. Identification of Credit Card Fraud (CCF) is a major challenge in financial security primarily due to concept drift. Fraudulent behaviors evolve dynamically, often making static models ineffective. This study proposes a drift-aware framework through the combined use of drift detection mechanism, a Memory-based Cuckoo Search (MCS) algorithm for feature selection, and the MAHIKIL oversampling technique, inspired by the chromosomal theory of inheritance. The predictive core of the framework is an online Long Short-Term Memory (LSTM) model, which adapts to data stream updates and responds to detected drift. Drift detection is achieved using ADWIN and EDDM, which handle sudden and gradual drifts respectively. Experimental evaluations on benchmark datasets, including the European Credit Cardholders Dataset (2013 and 2023) and a simulated transaction dataset, shows consistently high performance with accuracy, precision, recall, F1-score, and AUC values exceeding 99%. Overall, this work contributes a drift-resilient and feature-optimized approach that advances the development of adaptive fraud detection in real-time streaming.

Keywords: Concept Drift, Principal Component Analysis, Memory-Based Cuckoo Search, MAHAKIL, Online LSTM, Mahalanobis, Imbalance, Early Drift Detection Method, Adaptive Windowing.

1 Introduction

The rapid expansion of e-commerce, mobile banking, and digital payment systems has transformed the global financial landscape. In today's digital economy, credit cards have become a central component of both online and in-store transactions, offering speed, convenience, and efficiency to consumers worldwide [1]. As businesses transition from cash-based to cashless systems, credit cards are increasingly preferred by customers. However, this widespread adoption has also heightened exposure to fraudulent activities, resulting in substantial financial and reputational risks [2]. According to Xu et al. [3], credit card fraud (CCF) leads to billions of dollars in annual losses globally. CCF is a form of identity theft in which malicious actors conduct unauthorized

transactions using stolen credit card or account information. Between 2019 and 2020, identity theft cases rose by 113%, with credit card identity theft increasing by 44.6% and 365,597 cases involving new accounts reported within the same period [4]. Despite heavy investments in fraud prevention systems, existing solutions often fail to detect new and sophisticated attacks promptly. The dynamic nature of CCF therefore demands automated and adaptive detection systems capable of identifying potential fraud in real time [5]. Traditional rule-based detection methods have proven insufficient in addressing these evolving threats, as fraudsters continuously adapt their techniques to evade detection [6]. This has led to increased adoption of data-driven and adaptive approaches. Deep Learning (DL) has shown remarkable success in domains such as computer vision, natural language processing, and financial fraud detection. Techniques including Convolutional Neural Networks (CNNs), Artificial Neural Networks (ANNs), Recurrent Neural Networks (RNNs), Autoencoders, Generative Adversarial Networks (GANs), and Transformer architectures have been widely applied. Nevertheless, challenges persist—particularly concerning class imbalance, concept drift, feature relevance, false classification rates, and generalization to real-world conditions. A key issue in fraud detection is the highly imbalanced nature of CCF datasets, where fraudulent transactions represent a very small fraction of total transactions. Machine learning models tend to bias toward the majority (legitimate) class, achieving deceptively high accuracy while missing fraudulent cases [7]. Consequently, even a model that labels all transactions as legitimate can yield a misleading 99% accuracy [8]. To address this, data balancing methods such as Synthetic Minority Oversampling Technique (SMOTE) and its variants—ADASYN, Borderline-SMOTE, Majority Weighted Minority Oversampling Technique (MWMOTE), and Weighted Kernel-Based SMOTE—have been proposed. However, these methods can lead to over-generalization and high false positive rates [9], underscoring the need for more robust oversampling techniques capable of generating diverse yet realistic fraudulent samples. Beyond class imbalance, concept drift poses a significant challenge in maintaining long-term fraud detection accuracy. Concept drift occurs when the statistical properties of data evolve over time, altering the relationship between input features and output targets [10]. In financial systems, transaction behaviors change due to shifting customer habits, economic conditions, and evolving fraud strategies. Offline retraining is insufficient in such dynamic contexts, emphasizing the need for online or incremental learning [11]. Both gradual and abrupt drifts can lead to increased false positives and false negatives if not properly detected [12]. Without effective drift handling, models trained on outdated data lose predictive reliability and ultimately erode user trust. High-dimensional data also complicates model training and interpretation in fraud detection. Datasets often contain anonymized or redundant variables, which introduce the curse of dimensionality and overfitting [13]. Effective feature selection is therefore essential. Traditional filter and wrapper methods frequently struggle with the non-linear and complex interactions inherent in transactional data [14]. Nature-inspired metaheuristic algorithms have gained attention as powerful alternatives for feature optimization. Originating from the mid-1990s, “metaheuristics”—derived from “meta” (beyond) and “heuristic” (self-learning)—simulate natural or social processes to discover optimal solutions [15]. Algorithms such as the Grey Wolf Optimizer (GWO), Whale Optimization Algorithm (WOA), Bat Algorithm

(BA), Cuckoo Search Algorithm (CSA), and Ant Colony Algorithm (ACA) have been successfully applied to feature selection problems [16]. Nonetheless, each algorithm exhibits unique advantages and limitations in terms of convergence speed, exploration, and exploitation balance. To address these challenges, this study proposes an online learning-based Long Short-Term Memory (LSTM) framework integrated with concept drift detection for real-time CCF detection. A memory-based Cuckoo Search algorithm is adopted as a wrapper-based feature selection method to identify informative features while reducing redundancy and noise. Additionally, to counteract the class imbalance problem, the MAHAKIL oversampling technique—originally developed for software defect prediction—is incorporated. MAHAKIL uses chromosomal inheritance principles to generate diverse, high-quality synthetic samples, enhancing model learning without overfitting. The proposed model is validated using three datasets: two real-world credit card fraud datasets from Kaggle (2013 and 2023) and one synthetic dataset generated via Sparkov Data Generation. Model performance is evaluated through accuracy, precision, recall, F1-score, and Area Under the Curve (AUC), providing a comprehensive measure of both detection capability and robustness.

The major contributions of this study are threefold. First, it employs a memory-based Cuckoo Search algorithm for effective feature selection in high-dimensional, anonymized data. Second, it integrates Early Drift Detection Method (EDDM) and Adaptive Windowing (ADWIN) mechanisms to handle both gradual and sudden distributional shifts. Third, it develops an online LSTM architecture capable of learning from sequential transaction streams while adapting to evolving fraud behaviors. Collectively, these innovations advance adaptive learning in CCF detection and offer financial institutions a scalable, intelligent solution to mitigate fraud-related risks while maintaining operational trust and efficiency.

2 Related Works

The growing prevalence of credit card fraud (CCF), driven by the widespread adoption of digital payment technologies, has heightened the demand for intelligent and adaptive detection systems. Research in this field has evolved from rule-based models to machine learning and deep learning frameworks that capture behavioural, statistical, and temporal patterns to distinguish legitimate from fraudulent transactions. This section synthesizes key contributions across major analytical paradigms—probabilistic graphical models, classical machine learning, and deep learning—while emphasizing emerging hybrid and optimization-based frameworks that address long-standing challenges such as class imbalance, concept drift, and real-time scalability.

2.1 Probabilistic Graphical Models for Fraud Detection

Probabilistic graphical models formed the foundation for early CCF detection research. The Hidden Markov Model (HMM) proposed by Srivastava, Kundu, Shamik, & Arun K. [17] introduced a probabilistic framework for modelling transaction sequences, where transaction amounts served as observable variables and product categories as

hidden states. This model effectively captured spending patterns and deviations from user behaviour. Lucas et al. [18] enhanced detection performance by combining HMM-based sequence likelihoods with random forest classifiers, integrating temporal and static features for improved accuracy. Similarly, Gelivi, Katragadda, Gurram, Rajkumar, & Jabbar [19] applied Bayesian Belief Networks (BBNs) and Hidden Naïve Bayes (HNB) to capture inter-feature dependencies, yielding strong predictive power while retaining interpretability. These probabilistic approaches established a solid foundation for subsequent data-driven and hybrid models.

2.2 Classical Machine Learning Approaches

The emergence of machine learning shifted the focus from handcrafted probabilistic models to data-driven predictive systems. Classical algorithms such as logistic regression, decision trees, random forests, K-nearest neighbours (KNN), and support vector machines (SVMs) have been extensively studied for CCF detection. Decision tree-based classifiers are especially valued for interpretability and simplicity; Qawqzeh & Ashraf [20] achieved 99.97% accuracy and a strong F1-score using decision tree-based methods on transactional data. Random forests remain a dominant choice due to their robustness and ensemble nature. Afriyie et al. [21] demonstrated that ensemble learning consistently outperforms individual classifiers, achieving accuracies exceeding 96% on both synthetic and real-world datasets. Hybrid models have also shown great promise: Karunya, Samyuktha, Muralidharan, Brindha, & Muthu [22] improved KNN using logistic regression-derived distance metrics, while Aanjankumar & Poonkuntran, Singhai [23] demonstrated high precision using KNN for fraud identification. SVM-based frameworks, enhanced with feature selection and pruning techniques, further improved classification accuracy.

2.3 Deep Learning Approaches

Deep learning methods have redefined the landscape of fraud detection by automatically learning hierarchical representations from large-scale transactional data. Artificial Neural Networks (ANNs) demonstrated strong nonlinear modelling capabilities, achieving over 99% accuracy in studies by Asha & Suresh [24] and Rahman & Yao [25]. Convolutional Neural Networks (CNNs) have been adapted for CCF through effective feature extraction; Gambo, Zainal, & Kassim [26] achieved 99.82% accuracy using CNNs combined with ADASYN oversampling. Potluri et al. [27] integrated CNNs with Gradient Boosting Machines, reaching AUC scores near 0.99. Sequential deep learning methods such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) models [28] have demonstrated exceptional capability in modelling temporal transaction dependencies. LSTM models can learn long term dependencies and non-linear characteristic of a time series phenomenon [29]. Attention-based LSTMs [30] and bidirectional architectures [31] have further improved accuracy by focusing on contextual dependencies across transaction sequences. Gated Recurrent Units (GRUs), known for computational efficiency, have also achieved notable success; for instance, the Credit-Attention-GRU framework [32] produced an F1-score of

0.9835. In parallel, Generative Adversarial Networks (GANs) [33] have been used to generate realistic synthetic fraudulent transactions, addressing severe data imbalance issues. Transformer-based models [34] have further advanced the field by leveraging self-attention to capture long-range dependencies between transactions. Autoencoder-based approaches [35]—including variational and quantum variants—have been employed for unsupervised anomaly detection, achieving strong results based on reconstruction errors.

2.4 Emerging Directions and Optimization-Based Frameworks

Recent trends highlight the integration of optimization and hybrid algorithms for enhanced detection efficiency. Studies by Yajid et al. [36] and Ganji, Chaparala, & Sajja [37] have demonstrated that metaheuristic algorithms such as Cuckoo Search and Emperor Penguin Optimization significantly improve the accuracy and computational speed of deep learning models. Similarly, Atimbire, Appati, & Owusu [38] introduced Whale Optimization Algorithm with a logistic regression-based fitness function to select the most relevant heart diseases features from five datasets. This approach enhanced model performance of ten different machine learning models. In a related advancement, Ansah, Appati, Owusu, & Abdulai [39] proposed the Hybrid Seagull-Mayfly Optimisation algorithm to enhance energy efficiency and adaptive scheduling in wireless sensor networks, showcasing the versatility of hybrid heuristics beyond traditional data domains. Incremental and adaptive ensemble methods [40] have also been introduced to counteract concept drift and evolving fraud behaviours. Overall, the literature reflects a clear progression from probabilistic models to advanced hybrid and deep learning architectures. The ongoing challenge remains balancing model interpretability, adaptability, and real-time scalability—critical factors for reliable deployment in large-scale financial environments.

3 Methodology

This section outlines the methodological framework employed in the study, detailing the approach adopted to develop a drift-based framework for credit card fraud (CCF) detection. The methodology encompasses the preprocessing techniques applied to ensure data quality and balance, the feature selection strategies adopted to enhance model performance, and the subsequent model development process. Finally, the evaluation metrics used to assess the effectiveness and robustness of the proposed framework are discussed.

3.1 Proposed Methodology

This study proposes a robust and adaptive framework for credit card fraud (CCF) detection that integrates advanced data preprocessing, intelligent feature selection, and drift-aware online model learning. The framework is motivated by the highly dynamic nature of fraudulent activities, the pronounced class imbalance in CCF datasets, and the

need for models capable of adapting to evolving transactional patterns in real time. At its core, the proposed system employs an online Long Short-Term Memory (LSTM) architecture augmented with drift detection mechanisms to effectively manage temporal variations and changes in the statistical properties of data over time. Prior to model training, extensive preprocessing is conducted on all three datasets to ensure data quality and consistency. These procedures include normalization, label encoding, temporal feature handling, removal of redundant attributes, and class balancing using an oversampling strategy. To enhance model generalization and reduce redundancy, a swarm intelligence-based metaheuristic algorithm—namely, the Memory-Based Cuckoo Search (MCS)—is employed for feature selection. This algorithm efficiently identifies the most relevant and informative attributes, improving both the interpretability and performance of the learning model. The final phase of the methodology involves online-based model training using the LSTM classifier, integrated with drift detection mechanisms that trigger adaptive retraining when significant shifts in data distribution are detected. Model performance is comprehensively evaluated using key metrics such as accuracy, precision, F1-score, and Area Under the Receiver Operating Characteristic Curve (AUC). The overall methodological framework of the study is illustrated in Figure 1.

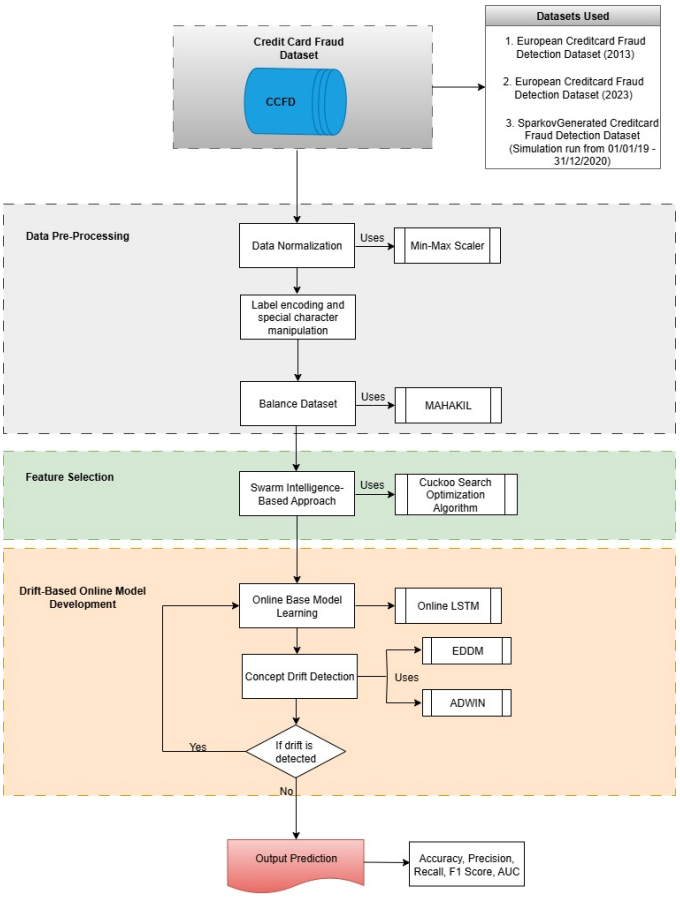


Fig 1. Proposed Methodology for Drift-Based Credit Card Fraud Detection

3.2 Data Preprocessing

The three datasets used in this study underwent rigorous preprocessing to ensure data quality, consistency, and model readiness. The preprocessing pipeline comprised four key steps: normalization, label encoding, redundant feature elimination, and feature transformation. Normalization was performed using the *min–max* scaling technique to rescale numerical features within a (0,1) range, improving model convergence and reducing feature dominance. In the first and second datasets, attributes such as *"time," "amount," "id,"* and *"amount"* were normalized, while in the synthetic dataset, features including *"merchant," "category," "amt," "city," "state," "lat," "long," "city_pop," "job," "unix_time," "merch_long," "hour," "weekday," "month,"* and *"day_of_year"* were adjusted to the same range. Categorical variables were transformed using label encoding, where each category was assigned a distinct integer value. This ensured compatibility with machine learning models while retaining data integrity. In the synthetically generated *Credit Card Transaction Fraud Dataset* (Sparkov),

attributes such as *"merchant," "category," "gender," "job," "state,"* and *"city"* were encoded—for example, *Male* and *Female* were represented as 1 and 0, respectively. Redundant features that contributed little predictive value or introduced multicollinearity were removed, particularly in the synthetic dataset. Attributes such as *"cc_num," "first," "last," "street," "zip," "dob,"* and *"trans_num"* were excluded to enhance efficiency and model interpretability. Finally, feature transformation was applied to the temporal attribute *"trans_date_trans_time."* From this, time-derived features (*"hour," "weekday," "month,"* and *"day_of_year"*) were generated, followed by sine and cosine transformations to preserve cyclical time patterns. The original timestamp was then dropped. This comprehensive preprocessing ensured balanced, noise-free, and well-structured datasets suitable for effective model training and real-time fraud detection.

3.3 Addressing Class Imbalance Using MAHAKIL

Class imbalance is crucial in classification tasks as it can lead to a biased model [41]. It is a critical challenge in Credit Card Fraud (CCF) detection, as fraudulent transactions typically represent less than 1% of total transactions in real-world datasets, such as the *European Cardholders Dataset (2013)* and the *Credit Card Transaction Fraud Detection Dataset*. Conventional oversampling approaches, including SMOTE and ADASYN, have been extensively applied to mitigate this issue. However, these techniques often lead to over-generalisation, generation of near-duplicate samples, and inflated false positive rates, which ultimately degrade classifier performance. To address these limitations, this study adopts MAHAKIL, a *diversity-based oversampling technique* grounded in the Chromosomal Theory of Inheritance [13]. As illustrated in Figure 2, MAHAKIL generates synthetic minority (fraudulent) instances through a biologically inspired process that mimics genetic inheritance. The algorithm begins with existing minority samples—treated as *parent nodes* (Node A1 and Node A2)—and employs the Mahalanobis distance to segregate them into two subgroups: *typical* and *atypical* instances. New synthetic "child" nodes (e.g., S01, S11, S12) are iteratively created by combining traits from each parent group, analogous to chromosomal recombination. Each subsequent generation of synthetic samples can, in turn, serve as parents for the next iteration, facilitating diversity propagation across multiple stages (Stage 0, Stage 1, Stage 2, … Stage n).

MAHAKIL, originally inspired by the *Chromosomal Theory of Inheritance* posits that offspring inherit 50% of their genetic material from each parent, thereby exhibiting traits from both while remaining distinct. Drawing from this principle, MAHAKIL conceptualises each fraudulent transaction as a *chromosome* composed of diverse *genes* (features). New synthetic instances are generated by recombining features from two distinct minority samples (parents). Unlike optimisation-driven genetic algorithms—which discard parent samples—MAHAKIL preserves all original instances while focusing on producing new, diverse samples that enhance class representation without redundancy. Algorithm 1 depict the MAHAKIL approach for oversampling.

Algorithm 1: Overview of MAHAKIL (N, P)

Input : Dataset of minority and majority class samples N; P_{fp} value of P
Output : Balanced dataset at a set P_{fp} value
Procedure Begin :

1. Split dataset N into arrays of minority classes N_{min} and majority class N
2. Compute the number of additional minority classes to be generated (T) to attain P_{fp} value P and $k = arraylength(N_{min})$;
3. X_{new}: array for generated sample, initialised to 0;
4. X_{newchk}: keeps count of number of synthetic samples generated;
5. Compute Mahalanobis distance (D^2) for N_{min},
$$D^2 = (x - \mu)^T \sum^{-1} (x - \mu)$$
where μ = mean and ∑ = covariance matrix;
6. Save D^2 matrix and its respective data samples in a decreasing order in N_{min}^{dist},
7. Find the midpoint or middle instance, i.e., $N_{mid} = k/2$;
8. Create two partitions from N_{min}^{dist} making N_{mid} the last record for partition 1 and $N_{mid} + 1$ the starting record for partition 2, i.e., $N_{bin1} = \{y_1, y_2, \dots, y_{mid}\}$ and $N_{bin2} = \{y_{mid+1}, y_{mid+2}, \dots, y_k\}$, where $y_i \in N_{min}^{dist}$,
9. For each $y_i \in N_{bin1}$ and $y_i \in N_{bin2}$, sequentially assign unique labels l_i from $i = 1, \dots, mid$;
10. ***for*** $i = 1$ ***to*** mid ***do***
Select instances y_a, y_b from N_{bin1} and N_{bin2} where label $l_i(y_a) = l_i(y_b)$;
Generate a minority class synthetic sample x by $average(y_a, y_b)$;
Add x to X_{new} and increment X_{newchk};
11. ***if*** $X_{newchk} < T$ ***then***
pair instances in X_{new} with instances in each parent bin N_{bin} and repeat steps 10 to 12 to create two different sets of $X_{new}^{(j)} (j = 1, 2)$;
if $X_{newchk} < T$ ***then***
pair each set of current X_{new} with its immediate parent set and label with predecessor set (i.e., set that contributed to generating current set) of immediate parent and repeat steps 10 to 12 of subsequent round;
else
assign all samples in X_{new} with the label of the minority class (fraud) and add to the dataset N;

End

Algorithm 1: MAHAKIL Approach for Oversampling.

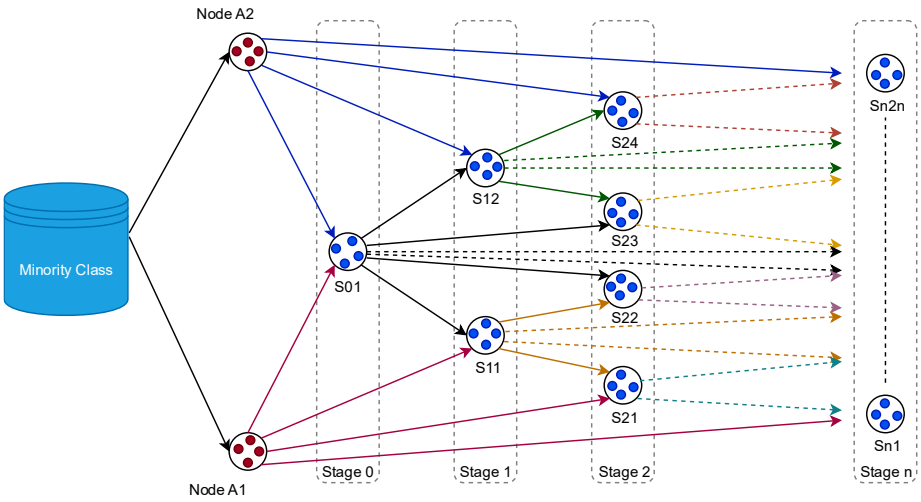


Fig 2. Conceptual illustration of the MAHAKIL oversampling technique.

The underlying intuition is that two distinct fraudulent instances, distant in feature space, can generate a synthetic sample that combines their characteristics. The process comprises three sequential phases: diversity measurement, data partitioning and pairing, and synthetic sample generation.

Phase 1: Diversity Measurement.

Diversity within the minority class is quantified using the *Mahalanobis distance*, a covariance-adjusted metric introduced by Prasanta Chandra Mahalanobis. For two p-dimensional instances $x = (x_1, x_2, x_{3,} \ldots\ x_n)^T$ and $y = (y_1, y_2, y_3, \ldots\ y_n)^T$, the Mahalanobis distance is defined as Eqn 1:

$$\mathrm{d}d(x, y) = \sqrt{(x - y)^T\, S^{-1}\, (x - y)} \tag{1}$$

where S^{-1} denotes the inverse of the covariance matrix. This measure accounts for inter-feature correlations, providing a more reliable assessment of diversity than Euclidean distance—especially in noisy or redundant transaction datasets. By ranking transactions in descending order of their Mahalanobis distances, the algorithm prioritises outliers and structurally diverse instances for subsequent pairing.

Phase 2: Data Partitioning and Pairing.

Fraudulent transactions are partitioned into two subsets: a *typical group* (samples closer to the class centroid) and an *atypical group* (samples farther away). The median of the Mahalanobis distance distribution serves as the partition threshold. Each transaction is sequentially labelled, and pairing is conducted between instances from the two subsets with matching labels. This binary pairing strategy ensures stable and generalisable synthetic data generation across multiple experimental runs.

Phase 3: Synthetic Sample Generation.

In the final phase, new fraud instances are synthesized by computing the *feature-wise mean* of paired samples drawn from the typical and atypical partitions (Node A1 and

Node A2). This mechanism contrasts sharply with local interpolation methods such as SMOTE, which rely on proximity-based synthesis. Instead, MAHAKIL intentionally combines *distant pairs* in the feature space, generating synthetic instances that are both diverse and representative. This enhances the minority class's coverage without introducing overlap with legitimate transactions.
By systematically propagating diversity, MAHAKIL ensures that the newly generated synthetic samples remain within the convex hull of the fraud class, preserving structural integrity while increasing representation density. This produces a well-balanced dataset that significantly improves classifier learning and robustness against concept drift.

3.4 Feature Selection Using Memory-Based Cuckoo Search (MCS)

Feature selection is a crucial step in machine learning that focuses on identifying and retaining the most relevant variables while removing redundant or non-informative ones. The effectiveness of deep learning models is highly dependent on the availability of massive datasets for training [42], which further underscores the need for effective feature selection to manage complexity and enhance model performance. Its importance lies in the direct influence of selected features on a model's predictive accuracy, interpretability, and generalisation capability. The three datasets employed in this study contained a large number of attributes, necessitating an effective and scalable feature selection mechanism. In recent years, metaheuristic algorithms have gained prominence in feature selection due to their capacity to efficiently navigate high-dimensional search spaces and overcome the limitations of deterministic or exhaustive methods. According to Jain, Jain, and Jangid [43], metaheuristics have become essential optimisation tools in modern research, especially with the growing prevalence of high-dimensional data. Unlike deterministic approaches, metaheuristics are inspired by natural processes such as biological evolution, swarm intelligence, and physical phenomena. Their strength lies in balancing exploration—searching diverse regions of the solution space to avoid local optima—and exploitation—refining promising solutions to converge toward optimal feature subsets. To enhance feature selection efficiency, this study employs a variant of the Cuckoo Search (CS) Algorithm, namely the Memory-Based Cuckoo Search (MCS) algorithm, to identify the most relevant features from the three CCF datasets.

Cuckoo Search (CS) Algorithm.
The Cuckoo Search (CS) algorithm, introduced by Yang and Deb [44], is a nature-inspired metaheuristic optimisation technique that simulates the parasitic reproduction strategy of cuckoo birds. Cuckoos lay their eggs in the nests of host birds; while some are detected and removed, others survive, symbolising superior solutions being retained through generations. Figure 3 conceptually depicts this analogy: the tree represents the solution space, the nests correspond to potential solutions, and the eggs represent candidate feature subsets. The acceptance or rejection of eggs reflects the evaluation and selection process within the optimisation cycle—where promising solutions are preserved while suboptimal ones are discarded.

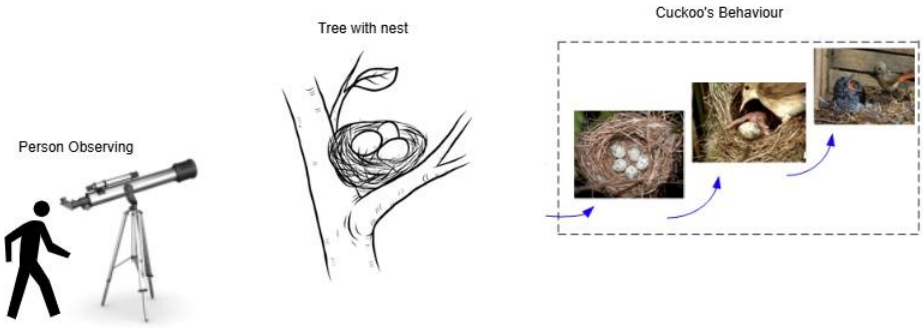


Fig 3. Conceptual illustration of Cuckoo Search Algorithm

Memory-Based Cuckoo Search (MCS) Algorithm.
To address the limitations of the standard CS algorithm, such as premature convergence and the loss of potentially valuable solutions in early iterations, this study uses the Memory-Based Cuckoo Search (MCS) algorithm. MCS introduces an adaptive memory mechanism that records and recalls historical solutions along with their fitness values. This allows the algorithm to revisit promising candidates, refine them, and prevent the loss of useful information. In figure 4, the process of the MCS algorithm is clearly shown. In this study, applying the MCS algorithm to the credit card fraud datasets improved accuracy while reducing the number of selected features compared to the standard CS. Algorithm 2 represents the MCS algorithm.

Algorithm 2: Memory-Based Cuckoo Search (MCS)

Input : Dataset (X, y); population size N; max iterations max_iter; discovery rate
p_a; Levy parameter β;
memory size m; abandon limit L; regeneration bias $\alpha \in (0,1]$; penalty weight
$\lambda \geq 0$
Output: Best solution $C^* \in \{0,1\}^d$ and fitness F^*
Encoding : Each nest $C \in \{0,1\}^d$ is a binary feature mask over d features
Initialise nests $\{C_1, \ldots, C_N\}$ uniformly at random in $\{0,1\}^d$
for $i \leftarrow 1$ *to* N ***do***
  $F_i \leftarrow$ EvaluateFitness (X, y, C_i, λ) // e.g., accuracy $-\lambda_d^{\#1s}$
$F^* \leftarrow max_i\ F_i$; $C^* \leftarrow C_{argmax_i} F_i$
Initialise memory $\mathcal{M} \leftarrow top\ m\ pairs\ \{(C_i, F_i)\}$ (sorted by F desc.)
Initialise abandon counter $a_i \leftarrow 0$ for all i
for $t \leftarrow 1$ max_*iter* ***do***
  // -- Random cuckoo selection and Levy-flight proposals --
  for $j \leftarrow 1$ *to* N ***do***
    pick cuckoo index $k \sim U\{1, \ldots, N\}$ and set $C \leftarrow C_k$.

$\Delta \leftarrow$ LevyFlight (β, d) // Mantegna: $\Delta = u/|u|^{1/\beta}$
$p \leftarrow \sigma(\Delta)$ // $\sigma(z) = \frac{1}{1+e^{-z}}$
sample flip mask $B \sim$ Bernoulli(p) (vectorised)
$C^* \leftarrow C \oplus B$ // bitwise XOR flips
$F_{C^*} \leftarrow$ EvaluateFitness (X, y, C^*, λ)
// -- Competition with a random next --
pick target index n$\sim U\{1, \dots, N\}$; $F_N \leftarrow F_n$.
if $F_{C^*} > F_N$ ***then***
  $C_n \leftarrow C^*$; $F_n \leftarrow F_{C^*}$; $a_n \leftarrow 0$
else
  $a_n \leftarrow a_n + 1$
// -- Abandon a fraction p_a of worst nest (or those reaching the limit) --
let W be indices of the worst $[p_a N]$ nests by fitness;
foreach $i \in W$ ***do***
  $a_n \leftarrow a_n + 1$
for $i \leftarrow 1$ *to* N ***do***
  if $a_i \geq L$ ***then***
    // Regeneration guided by elite memory
    choose (C_{elite}, F_{elite}) uniformly from $\mathcal{M}$;
    $\Delta \leftarrow$ LevyFlight (β, d); dir $\leftarrow C_{elite} - C_i$;
    $z \leftarrow \Delta + \alpha \cdot$ dir; $p \leftarrow \sigma(z)$; sample $B \sim$ Bernoulli(p);
    $C^* \leftarrow C \oplus B$; $F_i \leftarrow$ EvaluateFitness (X, y, C_i, λ); $a_i \leftarrow 0$
// -- Elite memory update --
$\mathcal{M} \leftarrow Top_m(\mathcal{M} \cup \{(C_i, F_i)\}_{i=1}^{N})$
// -- Track best-so-far --
find $i^* \in \arg max_i \ F_i$;
if $F_{i^*} > F^*$ ***then***
  $F^* \leftarrow F_{i^*}$; $C^* \leftarrow C_{i^*}$
return (C^*, F^*)

Algorithm 2: Memory-Based Cuckoo Search Algorithm.

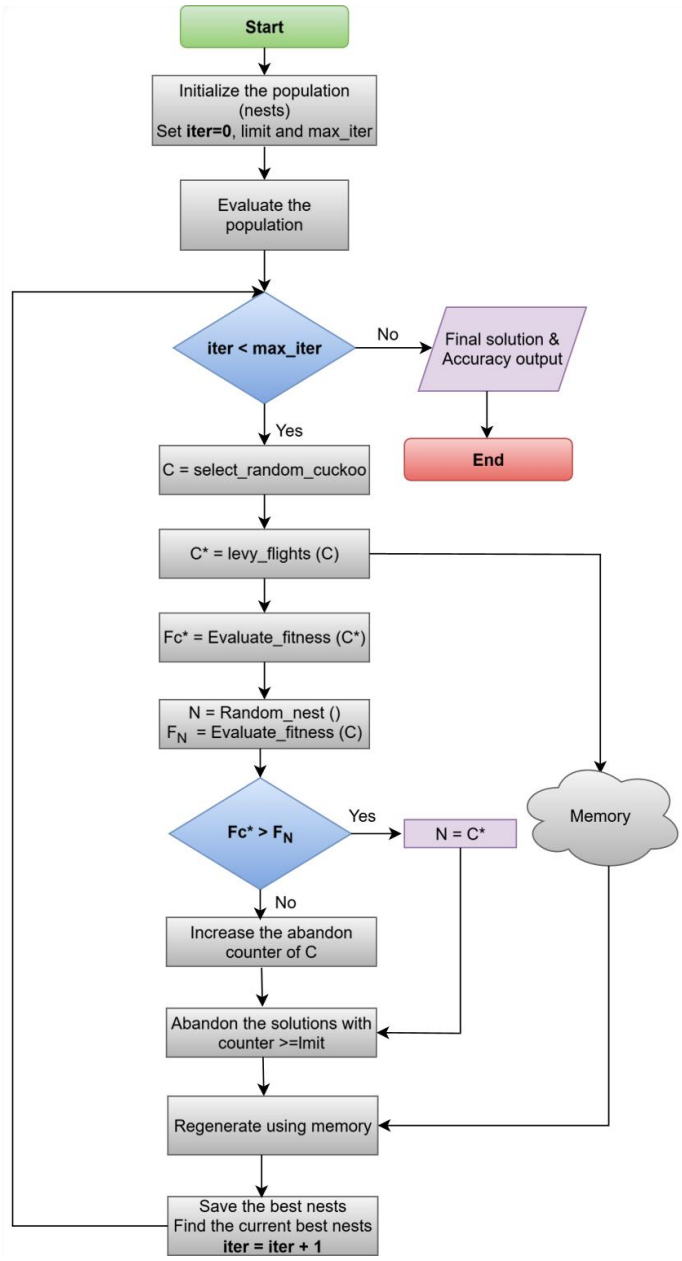


Fig 4. Pipeline of the MCS algorithm

Parameter Configuration.
The major parameters used in the MCS algorithm are presented in Table 1.

Table 1. MCS algorithm assigned parameters and values

Parameters	Value
Population size (n_nests)	20
Maximum iterations (max_iter)	30
Discovery probability ($\boldsymbol{Pa}$)	0.25
Levy flight step (β)	1.5
Penalty weight	0.01
Memory size	5
Abandon limit	5

Algorithmic Workflow.

Initialisation. The process begins with the random generation of a population of nests, where each nest represents a potential subset of features. In the European Cardholders Dataset (2013), for instance, features such as *Time*, *Amount*, and *V1–V28* were considered. Random initialisation promotes diversity and prevents premature convergence by ensuring the search begins from varied points in the solution space.

Population Evaluation. Each candidate nest is evaluated using a fitness function based on classification accuracy. A logistic regression classifier is trained using the selected features, and its performance is measured using Eqn 2 on a randomised 70–30 train-test split.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \tag{2}$$

To promote model parsimony, a penalty term discourages the selection of excessive features, forming the final fitness function in Eqn 3.

$$F = \text{Acc} - \lambda\frac{k}{d} \tag{3}$$

where k is the number of selected features, d the total number of features, and λ the penalty weight.

Random Cuckoo Selection and Lévy Flights. A cuckoo is randomly selected, and a new candidate solution is generated using Lévy flights, a type of random walk characterised by small steps interspersed with occasional long jumps. This dynamic balances local search and global exploration. The step size is computed with Eqn 4:

$$step = \frac{\mu}{|v|^{1/\beta}} \tag{4}$$

where $u \sim N(0, \sigma_u^2)$, $v \sim N(0,\ 1)$, and β is the Levy distribution parameter, while σ_u is computed using Mantegna's algorithm to make sure the correct scaling of the Levy distribution is captured. The step is converted to a flip probability using the sigmoid function in Eqn 5.

$$\text{P(flip)} = \frac{1}{1+ e^{-step}} \tag{5}$$

ensuring controlled feature inclusion or exclusion within binary feature masks.

Selection and Replacement. If a newly generated solution (F_{C^*}) outperforms a randomly chosen nest (F_N), it replaces the latter, and its abandonment counter resets to zero. Otherwise, the counter increases, gradually marking poor-performing solutions for replacement.
Abandonment and Regeneration. When a nest's abandonment counter exceeds a predefined limit, it is regenerated using elite memory guidance, rather than pure randomness. The regeneration process is given by Eqn 6:

$$C_{\text{new}} = C_{\text{old}} \oplus \text{flip}\left(\sigma\left(\text{step} + 0.3(C_{\text{elite}} - C_{\text{old}})\right)\right) \tag{6}$$

where C_{elite} represents a high-performing solution from memory, σ is the sigmoid function, and $\oplus$ denotes bitwise flipping.
Memory Update. After each generation, the elite memory is updated by combining the current nests and their fitness values with previously stored elite solutions. Only the top m solutions are retained in Eqn 7:

$$M^{(t-1)} = \text{Top}_m\left(M^{(t)} \cup N^{(t)}\right) \tag{7}$$

This ensures that historically successful feature subsets, such as *Amount* or *Time*, which consistently enhance fraud detection, are preserved for reuse.
C^*_{best}, and its corresponding fitness value F^*_{best}.
Iteration Update and Termination. The algorithm iteratively updates the best current solution (C^*_{best}) and its corresponding fitness value (F^*_{best}). The process continues until the maximum number of iterations (max_iter=30) is reached, at which point the best feature subset and its performance score are recorded.

Stability-Based Feature Selection.
Given the stochastic nature of MCS, a stability-based refinement was employed to ensure robustness and reproducibility. The algorithm was executed for $n_{runs} = 5$ independent runs, and feature selection frequency across runs was computed as Eqn 8.

$$freq(f_j) = \frac{1}{n_{runs}} \sum_{i=1}^{n_{runs}} I\left(fj \in C^{(i)}\right) \tag{8}$$

where $I(\cdot)$ is the indicator function. Features selected in at least 80% of runs were retained in the *European Cardholders Dataset (2013)*, while a threshold of 60% was applied for the *European Cardholders Dataset (2023)* and the *Credit Card Transaction Fraud Detection Dataset*. This ensured that only features consistently contributing to model performance across multiple trials were included in the final subset.

3.5 Drift-Based Online LSTM Architecture for Adaptive Credit Card Fraud Detection

The Drift-Based Online Long Short-Term Memory (LSTM) architecture is developed to effectively manage sequential transaction streams in real time, particularly within dynamic environments where underlying data distributions evolve—a phenomenon

known as concept drift. Unlike conventional batch-trained models, which assume static data distributions, the proposed architecture updates its parameters continuously as new transaction records arrive. To ensure robustness against such distributional changes, two complementary drift detection mechanisms—Early Drift Detection Method (EDDM) and Adaptive Windowing (ADWIN)—are employed to monitor and log drift occurrences during online learning.

Concept Drift

Concept drift refers to temporal changes in the underlying distribution that generates the observed data [45]. In credit card fraud (CCF) detection, where transaction behaviours evolve over time, this phenomenon is particularly critical. Fraudulent strategies observed historically may differ significantly from those in contemporary datasets, as perpetrators continuously refine their tactics to bypass detection systems. Consequently, models trained solely on historical data often degrade in performance when applied to new transactions, underscoring the necessity for adaptive, drift-aware learning mechanisms. Given the heterogeneity of the three CCF datasets used in this study, multiple forms of concept drift—both gradual and abrupt—are anticipated. To ensure reliable detection across these scenarios, this study integrates two drift detection algorithms with complementary strengths. EDDM is incorporated for its sensitivity to gradual shifts, where fraudulent behaviour evolves progressively, whereas ADWIN is employed for its ability to identify abrupt, discontinuous changes in data distributions. The dual integration enables the system to respond effectively to both gradual and sudden fraud behaviour shifts, thereby maintaining predictive accuracy over time.

Early Drift Detection Method (EDDM)

The EDDM mechanism is utilised to detect gradual drifts in the CCF datasets during online learning. Traditional drift detectors, which rely solely on monitoring classification error rates, often fail in scenarios where data evolves slowly. EDDM addresses this limitation by analysing the distances between consecutive misclassifications, rather than the error frequency itself. For each misclassified transaction, EDDM computes the number of correctly classified transactions since the previous error—termed the *error distance*. Over time, the mean error distance (μ_t) and its standard deviation (σ_t) are maintained, and the stability metric ($\mu_t + 2\sigma_t$) is compared against the baseline maximum ($(\mu + 2\sigma)_{max}$) established during the model's initial learning phase. Drift detection is guided by the following relationship in Eqn 9.

$$p_t = \frac{\mu_t + 2\sigma_t}{(\mu + 2\sigma)_{max}} \tag{9}$$

when (p_t) falls below a predefined warning threshold, EDDM signals a potential drift; a further decline beyond a stricter threshold confirms drift occurrence. In this framework, confirmed drift events trigger a reset and reinitialisation of the Online LSTM model, enabling rapid adaptation to newly emerging fraudulent behaviours.

Adaptive Windowing (ADWIN)

ADWIN complements EDDM by detecting abrupt concept drifts—sudden changes in fraud patterns that can result from new attack strategies, such as large-scale coordinated breaches or card-not-present (CNP) fraud schemes. ADWIN maintains a variable-length window (W) containing recent prediction results and dynamically adjusts its size

in response to changes in data distribution. At each time step, ADWIN tests whether (W) can be partitioned into two sub-windows, (W_1) and (W_2). A drift is declared when the absolute difference between their mean values exceeds a statistically defined threshold (ϵ) in Eqn 10.

$$|\mu_{W1} - \mu_{W2}| > \epsilon \tag{10}$$

The threshold (ϵ) is derived using the Hoeffding bound in Eqn (11).

$$\epsilon = \sqrt{\frac{1}{2m} \ln\left(\frac{4}{\delta}\right)} \tag{11}$$

where (m) denotes the harmonic mean of the window sizes and (δ) represents the confidence parameter controlling the false-positive rate. When this condition is satisfied, ADWIN discards outdated samples (window (W_1)) and retains only the most recent observations (W_2), ensuring that model updates are based on current transaction characteristics. This dynamic adjustment is particularly beneficial in fraud detection, as it enables the system to respond to sudden shifts in fraudulent activity, such as transitions from isolated high-value frauds to numerous low-value microtransactions. When no drift is detected, ADWIN expands its window, thereby enhancing the model's statistical stability by leveraging longer sequences of consistent transactions.

Online LSTM Architecture

The LSTM network forms the adaptive core of this architecture and is implemented within an online learning framework. Unlike traditional batch models, the Online LSTM incrementally updates its parameters as each transaction arrives, making it well-suited for non-stationary data environments. The LSTM architecture, illustrated in Figure 5, employs a gated memory mechanism to capture both short- and long-term temporal dependencies in transaction sequences. Given an input transaction vector (x_t) at time step (t), the hidden state (h_t) and cell state (c_t) are updated according to the standard LSTM recurrence in Eqn 12.

$$(h_t, c_t) = LSTM(x_t, h_{t-1}, c_{t-1}) \tag{12}$$

The gating functions regulate the flow of new and historical information, enabling the model to selectively retain relevant past patterns while discarding obsolete ones. After each transaction, the forward pass is followed by a backward pass and an incremental parameter update using the Adam optimiser. This ensures that the network continuously refines its parameters in response to the most recent data, rather than waiting for batch retraining. By integrating EDDM and ADWIN within the Online LSTM framework, the system achieves both adaptability and resilience. Detected drift events trigger reinitialisation and retraining on the latest transaction indices, allowing the model to remain synchronised with evolving data distributions and maintain high predictive performance in real-time CCF detection.

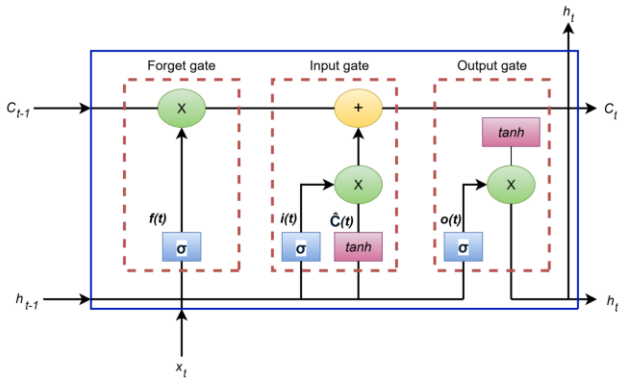


Fig 5. Long Short-Term Memory (LSTM) architecture.

3.6 Performance Evaluation Metrics

The effectiveness of the proposed framework was assessed using standard classification measures—accuracy, precision, recall, F1-score, and AUC—commonly applied in binary classification and fraud detection studies.

Accuracy evaluates the proportion of correctly classified transactions, expressed as:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{13}$$

where TP and TN represent correctly identified fraudulent and legitimate transactions, respectively, while FP and FN denote incorrect classifications. However, in highly imbalanced datasets, accuracy alone can be misleading.

To provide deeper insight, precision and recall in Eqn 14 were employed. Precision measures the proportion of transactions predicted as fraud that are truly fraudulent:

$$Precision = \frac{TP}{TP+FP} \tag{14}$$

while recall (or sensitivity) in Eqn 15 measures the proportion of actual fraud cases correctly identified:

$$Recall = \frac{TP}{TP+FN} \tag{15}$$

High precision reduces false alarms, and high recall minimises missed detections—both crucial in fraud detection where errors carry financial and operational risks.

To balance these trade-offs, the F1-score in Eqn 16, defined as the harmonic mean of precision and recall, was used.

$$F1\ score = \frac{2\ (Precision\ x\ Recall)}{Precision+Recall} \tag{16}$$

Finally, the Area Under the ROC Curve (AUC) evaluates the model's discriminatory power. With True Positive Rate (TPR) and False Positive Rate (FPR) defined as Eqn 17.

$$FPR = \frac{FP}{FP+TN} \tag{17}$$

the AUC is calculated as Eqn 18.

$$AUC = \int_0^1 TPR(FPR)d(FRP) \tag{18}$$

An AUC close to 1.0 signifies strong separation between fraudulent and legitimate transactions. Collectively, these metrics ensure a balanced, robust, and interpretable evaluation of the framework's performance in real-world credit card fraud detection settings.

4 Results and Discussions

4.1 Datasets Description

To evaluate the robustness and generalizability of the proposed credit card fraud (CCF) detection framework, three heterogeneous datasets were employed: the European Credit Cardholders Datasets (2013 and 2023) and the Credit Card Transactions Fraud Detection Dataset. These datasets provide diverse transactional characteristics, temporal spans, and class imbalance scenarios, enabling comprehensive model assessment.

European Credit Cardholders Dataset (2013)

The first dataset, sourced from Kaggle, contains 284,807 transactions from European cardholders in September 2013, of which 494 are fraudulent, representing 0.172% of all transactions. This extreme imbalance highlights the challenges of CCF detection. All features underwent Principal Component Analysis (PCA), producing 30 attributes: V1–V28 (anonymized principal components), Time, and Amount. The binary Class column indicates fraudulent (1) and legitimate (0) transactions. This dataset serves as a benchmark for imbalanced classification. The class distribution is shown in Figure 6.

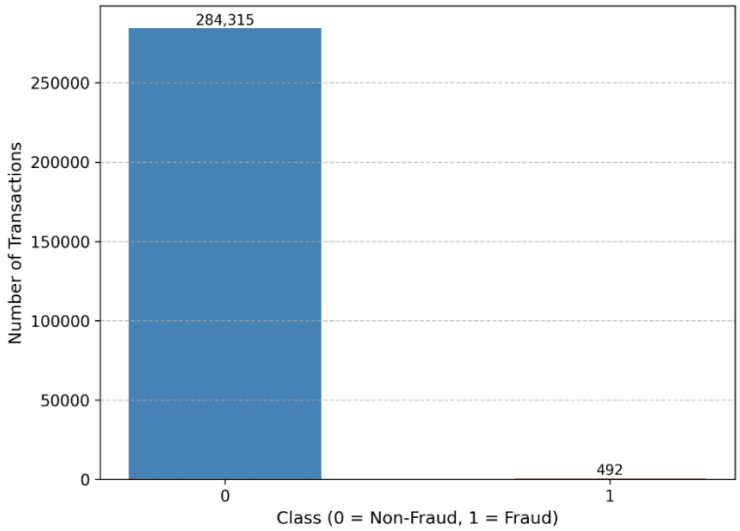


Fig 6. Distribution of transaction records categorized as Non-Fraud (0) and Fraud (1) with the European Credit Cardholders Dataset (2013).

European Credit Cardholders Dataset (2023)

The second dataset, also from Kaggle, comprises 568,630 transactions from 2023 and exhibits a balanced distribution between fraud and non-fraud instances. It includes 30 numerical features—V1–V28 (anonymized attributes), Amount, and ID—with the binary Class column. All features were converted to numerical representations to facilitate model training. The class distribution is shown in Figure 7, and detailed feature descriptions are summarized in Table 2.

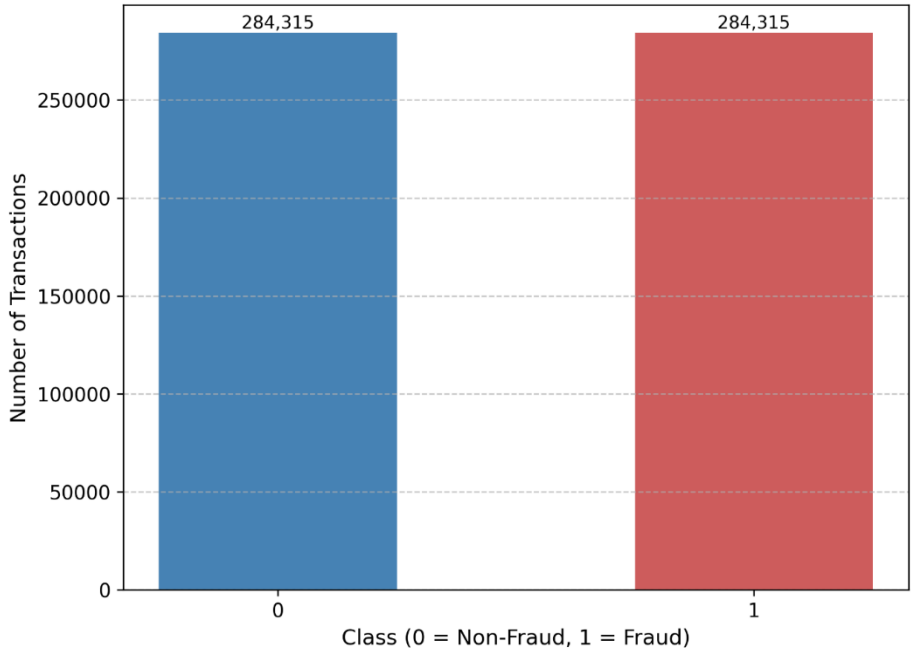


Fig 7. Distribution of transaction records categorized as Non-Fraud (0) and Fraud (1) with the European Credit Cardholders Dataset (2023).

Table 2. Description of features in the European Cardholders Dataset (2023)

Features	Description
id	Unique identifier for each transaction performed
V1-V28	Anonymized features representing the various transaction attributes (e.g., time, location, etc.)
Amount	The money involved in the transaction
Class	Binary label indicating whether the transaction is fraudulent (1) or not (0)

Credit Card Transactions Fraud Detection Dataset

The third dataset, generated synthetically using the Sparkov Data Generation Tool, contains 1,296,675 transactions spanning January 2019 to December 2022. It includes 22 features, covering temporal, spatial, demographic, and merchant-specific attributes, such as trans_date_trans_time, merchant, category, amt, lat/long, gender, job, and city_pop. The binary target variable is_fraud identifies fraudulent (1) and legitimate (0) transactions. Of these, 7,506 are fraudulent, reflecting the natural class imbalance typical of real-world datasets. Feature descriptions are detailed in Table 3, and the class distribution is illustrated in Figure 8.

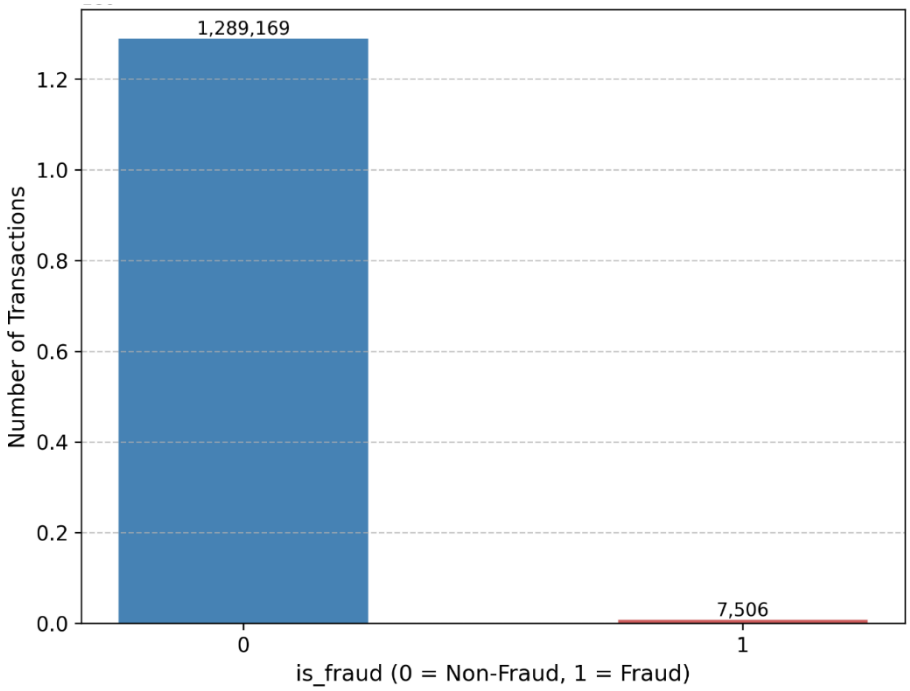


Fig 8. Distribution of records categorised as Non-Fraud (0) and Fraud (1) within the Credit Card Transactions Fraud Detection Dataset.

Table 3. Description of features in the Credit Card Transactions Fraud Detection Dataset

Features	Description
trans_date_trans_time	Timestamp of the transaction (date and time)
cc_num	Unique credit card number identifier
merchant	Name or identifier of the merchant where the transaction occurred
category	Merchant category (e.g., groceries, travel, shopping)
amt	Transaction amount in monetary units
first	First name of the cardholder
last	First name of the cardholder
gender	Gender of the cardholder, i.e. Male or female
street	Street address of the cardholder
city	City of the cardholder
state	State of the cardholder
zip	Zip code of the cardholder
lat	Latitude coordinate of the cardholder's address
long	Longitude coordinate of the cardholder's address
city_pop	Population of the city in which the cardholder resides
job	Occupation of the cardholder
dob	Date of birth of cardholder

trans_num	Unique transaction identifier
unix_time	Transaction time in UNIX format (seconds since epoch)
merch_lat	Latitude coordinate of the merchant's location
merch_long	Longitude coordinate of the merchant's location
is_fraud	Binary class label indicating fraud (1) and non-fraud (0)

4.2 Feature Selection Outcome

The Table 4 provides insights into the final accuracy, selected features, and removed features across the three datasets used in this study.

Table 4. Datasets with corresponding final accuracy, selected features, and removed features after MCS algorithm.

Datasets	Final Accuracy	Selected Features	Removed Features
European Credit Cardholders Dataset (2013)	0.9829	V1, V3, V4, V5, V8, V10, V11, V12, V14, V16, V17, V25, V27, V28	Time, V2, V6, V7, V9, V13, V15, V18, V19, V20, V21, V22, V23, V24, V26, Amount
European Credit Cardholders Dataset (2023)	0.9982	id, V4, V11, V12, V14, V18, V19, V24	V1, V2, V3, V5, V6, V7, V8, V9, V10, V13, V15, V16, V17, V20, V21, V22, V23, V25, V26, V27, V28, Amount
Credit Card Transactions Fraud Detection Dataset	0.8965	amt, unix_time, hour, hour_sin, hour_cos	merchant, category, gender, city, state, lat, long, cit_pop, job, merch_long, weekday, month, day_of_year, weekday_sin, weekday_cos

Figures 9-11 show bar charts of selected features after multiple runs across the three datasets. The red dashed line represents the threshold (τ = 0.8, 0.6). Features with bars above the line were excluded, while those below or directly on the line were retained.

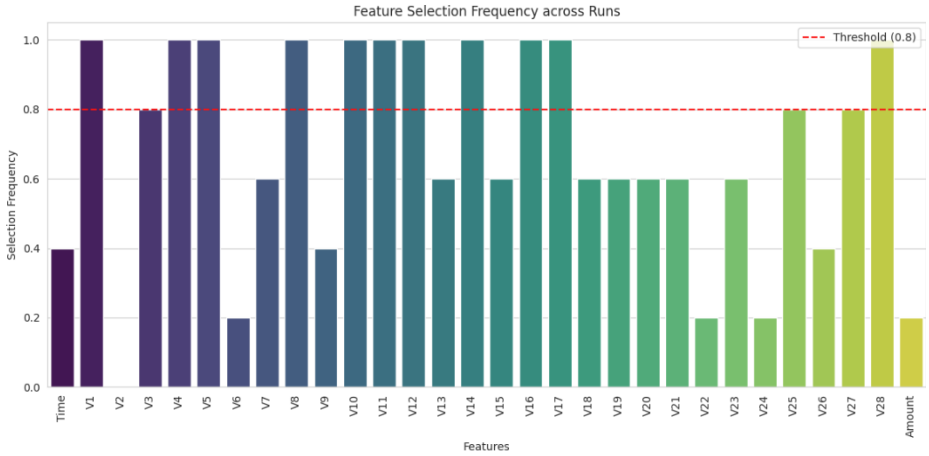


Fig 9. Feature selection frequency on the European Credit Cardholders Dataset (2013) with threshold $\tau = 0.8$

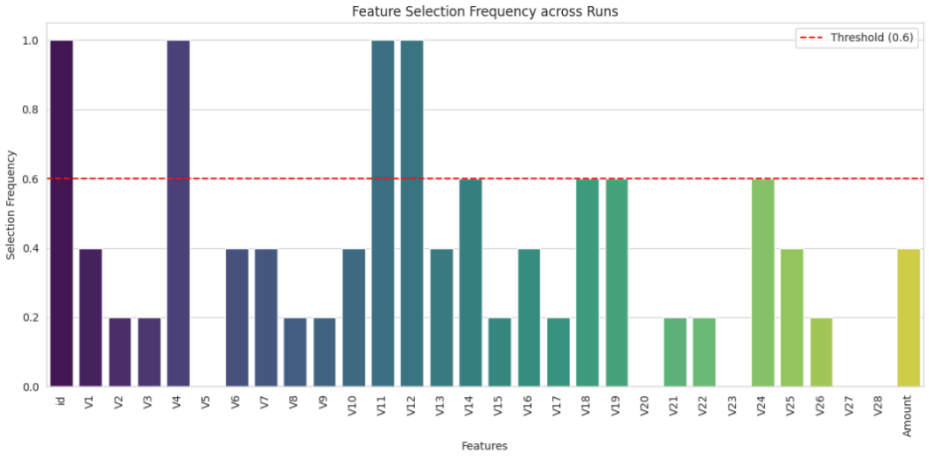


Fig 10. Feature selection frequency on the European Credit Cardholders Dataset (2023) with threshold $\tau = 0.6$

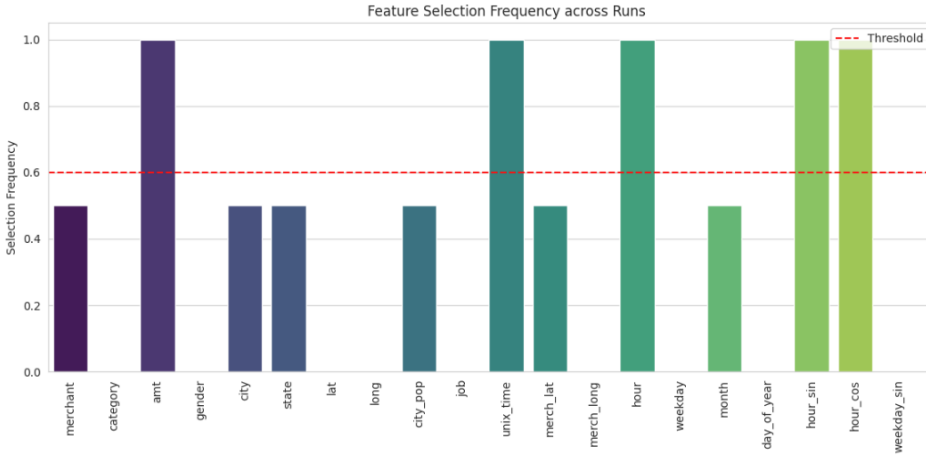


Fig 11. Feature Selection Frequency on the Credit Card Transactions Fraud Detection Dataset with Threshold $\tau = 0.6$

4.3 Model Performance: European Credit Cardholders Dataset (2013)

The 2013 dataset contained 30 features, including Time, Amount, and anonymized components V1–V28. Feature selection using the MCS algorithm reduced this to 14 relevant features, which were input to the drift-aware Online LSTM. Model performance was evaluated every 10,000 transactions using Accuracy, Precision, Recall, F1-score, and AUC. EDDM and ADWIN detected concept drifts at indices 94,333, 135,824, 183,359, 223,000 (EDDM) and 227,967 (ADWIN), triggering model reinitialization to adapt to new transaction patterns. Table 5 demonstrates that the proposed framework maintains predictive performance and rapidly adjusts to evolving fraudulent behavior.

Table 5. Results of the Online LSTM integrated with EDDM on the European Credit Cardholders Dataset (2013)

Evaluation Step	Accuracy	Precision	Recall	F1-score	AUC
10,000	0.9982	0.3333	0.5000	0.4000	0.7494
20,000	0.9988	0.5667	0.5862	0.5763	0.7928
30,000	0.9991	0.7083	0.7083	0.7083	0.8539
40,000	0.9991	0.7586	0.6875	0.7213	0.8436
50,000	0.9992	0.7794	0.6883	0.7310	0.8440
60,000	0.9993	0.7895	0.6897	0.7362	0.8447
70,000	0.9992	0.7753	0.6699	0.7188	0.8348
80,000	0.9993	0.7802	0.6698	0.7208	0.8348
90,000	0.9993	0.8056	0.6905	0.7436	0.8451
100,000	0.9992	0.8187	0.7254	0.7692	0.8625
110,000	0.9992	0.8101	0.7214	0.7632	0.8605

120,000	0.9992	0.8077	0.7171	0.7597	0.8584
130,000	0.9993	0.8032	0.7156	0.7569	0.8577
140,000	0.9992	0.7718	0.7067	0.7378	0.8532
150,000	0.9992	0.7746	0.6992	0.7350	0.8494
160,000	0.9992	0.7818	0.6992	0.7382	0.8494
170,000	0.9993	0.7870	0.6988	0.7403	0.8493
180,000	0.9993	0.7910	0.7070	0.7466	0.8533
190,000	0.9992	0.7575	0.6952	0.7250	0.8474
200,000	0.9992	0.7711	0.6997	0.7337	0.8497
210,000	0.9992	0.7782	0.7059	0.7403	0.8528
220,000	0.9992	0.7763	0.6960	0.7340	0.8479
230,000	0.9992	0.9709	0.9494	0.9600	0.9745
240,000	0.9992	0.9945	0.9902	0.9924	0.9950
250,000	0.9992	0.9970	0.9946	0.9958	0.9971
260,000	0.9993	0.9979	0.9963	0.9971	0.9980
270,000	0.9993	0.9984	0.9971	0.9978	0.9984
280,000	0.9993	0.9987	0.9977	0.9982	0.9987
290,000	0.9993	0.9989	0.9981	0.9985	0.9989
300,000	0.9994	0.9991	0.9983	0.9987	0.9990
310,000	0.9994	0.9992	0.9985	0.9989	0.9991
320,000	0.9994	0.9993	0.9987	0.9990	0.9992
330,000	0.9994	0.9993	0.9988	0.9991	0.9993
340,000	0.9994	0.9994	0.9989	0.9992	0.9993
350,000	0.9995	0.9994	0.9990	0.9992	0.9994
360,000	0.9995	0.9995	0.9991	0.9993	0.9994
370,000	0.9995	0.9995	0.9992	0.9993	0.9994
380,000	0.9995	0.9996	0.9992	0.9994	0.9995
390,000	0.9995	0.9996	0.9993	0.9994	0.9995
400,000	0.9995	0.9996	0.9993	0.9995	0.9995
410,000	0.9995	0.9996	0.9993	0.9995	0.9995
420,000	0.9996	0.9996	0.9994	0.9995	0.9995
430,000	0.9996	0.9997	0.9994	0.9995	0.9996
440,000	0.9996	0.9997	0.9994	0.9996	0.9996
450,000	0.9996	0.9997	0.9995	0.9996	0.9996
460,000	0.9996	0.9997	0.9995	0.9996	0.9996
470,000	0.9996	0.9997	0.9995	0.9996	0.9996
480,000	0.9996	0.9997	0.9995	0.9996	0.9996
490,000	0.9996	0.9997	0.9995	0.9996	0.9996
500,000	0.9996	0.9998	0.9996	0.9997	0.9996
510,000	0.9996	0.9998	0.9996	0.9997	0.9996
511,767	0.9996	0.9998	0.9996	0.9997	0.9996

The graphs presented below illustrate the variation in Accuracy, Precision, Recall, F1-score, and AUC as successive samples are introduced into the model during the evaluation process.

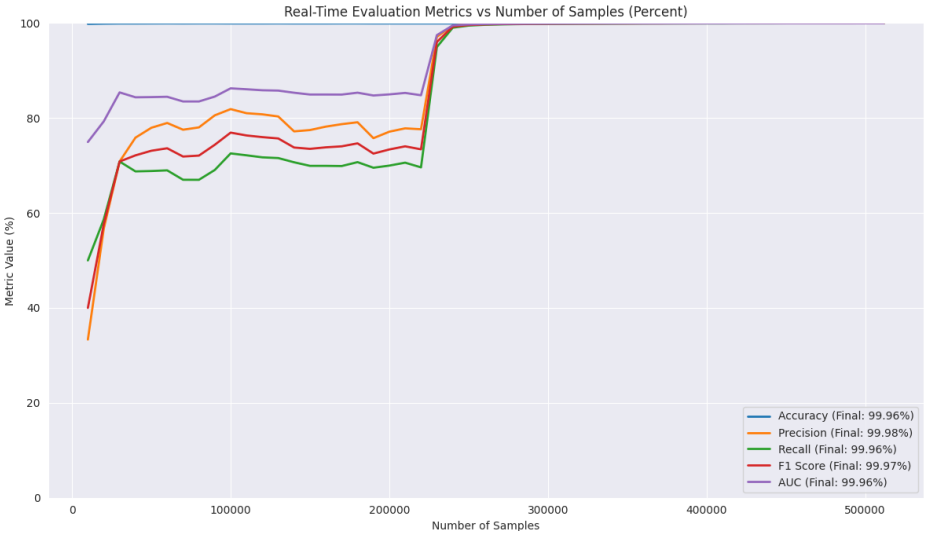


Fig 12. Real-Time Accuracy, Precision, Recall, F1 Score, and AUC progression of the model with EDDM across the European Credit Cardholders Dataset (2013).

The Table 6 also presents the results obtained at each evaluation step when the Online LSTM model was integrated with Adaptive Windowing (ADWIN).

Table 6. Results of the Online LSTM integrated with ADWIN on the European Credit Cardholders Dataset (2013).

Evaluation Step	Accuracy	Precision	Recall	F1-score	AUC
10,000	0.9988	0.5000	0.5000	0.5000	0.7497
20,000	0.9991	0.7200	0.6207	0.6667	0.8102
30,000	0.9993	0.8140	0.7292	0.7692	0.8644
40,000	0.9993	0.8491	0.7031	0.7692	0.8515
50,000	0.9993	0.8438	0.7013	0.7660	0.8505
60,000	0.9994	0.8472	0.7011	0.7673	0.8505
70,000	0.9993	0.8148	0.6408	0.7174	0.8203
80,000	0.9993	0.8193	0.6415	0.7196	0.8207
90,000	0.9994	0.8400	0.6667	0.7434	0.8332
100,000	0.9992	0.8503	0.7358	0.7889	0.8678
110,000	0.9992	0.8391	0.7264	0.7787	0.8631
120,000	0.9993	0.8409	0.7220	0.7769	0.8609
130,000	0.9993	0.8352	0.7204	0.7735	0.8601

140,000	0.9993	0.8299	0.7156	0.7685	0.8577
150,000	0.9993	0.8317	0.7119	0.7671	0.8558
160,000	0.9993	0.8373	0.7114	0.7692	0.8556
170,000	0.9994	0.8371	0.7143	0.7708	0.8570
180,000	0.9994	0.8376	0.7179	0.7732	0.8589
190,000	0.9994	0.8427	0.7158	0.7741	0.8578
200,000	0.9994	0.8491	0.7188	0.7785	0.8593
210,000	0.9994	0.8540	0.7245	0.7839	0.8621
220,000	0.9994	0.8520	0.7173	0.7789	0.8586
230,000	0.9994	0.9820	0.9557	0.9686	0.9777
240,000	0.9994	0.9966	0.9914	0.9940	0.9956
250,000	0.9994	0.9981	0.9953	0.9967	0.9975
260,000	0.9994	0.9987	0.9967	0.9977	0.9983
270,000	0.9995	0.9990	0.9975	0.9983	0.9987
280,000	0.9995	0.9992	0.9980	0.9986	0.9989
290,000	0.9995	0.9993	0.9983	0.9988	0.9991
300,000	0.9995	0.9994	0.9985	0.9990	0.9992
310,000	0.9995	0.9995	0.9987	0.9991	0.9993
320,000	0.9995	0.9995	0.9989	0.9992	0.9993
330,000	0.9996	0.9996	0.9990	0.9993	0.9994
340,000	0.9996	0.9996	0.9991	0.9993	0.9994
350,000	0.9996	0.9997	0.9997	0.9994	0.9995
360,000	0.9996	0.9997	0.9992	0.9994	0.9995
370,000	0.9996	0.9997	0.9993	0.9995	0.9995
380,000	0.9996	0.9997	0.9993	0.9995	0.9996
390,000	0.9996	0.9997	0.9993	0.9995	0.9996
400,000	0.9996	0.9998	0.9994	0.9996	0.9996
410,000	0.9996	0.9998	0.9994	0.9996	0.9996
420,000	0.9996	0.9998	0.9994	0.9996	0.9996
430,000	0.9997	0.9998	0.9995	0.9996	0.9996
440,000	0.9997	0.9998	0.9995	0.9997	0.9997
450,000	0.9997	0.9998	0.9995	0.9997	0.9997
460,000	0.9997	0.9998	0.9995	0.9997	0.9997
470,000	0.9997	0.9998	0.9996	0.9997	0.9997
480,000	0.9997	0.9998	0.9996	0.9997	0.9997
490,000	0.9997	0.9998	0.9996	0.9997	0.9997
500,000	0.9997	0.9998	0.9996	0.9997	0.9997
510,000	0.9997	0.9999	0.9996	0.9997	0.9997
511,767	0.9997	0.9999	0.9996	0.9997	0.9997

The graphs presented below illustrate the variation in Accuracy, Precision, Recall, F1-score, and AUC as successive samples are introduced into the model during the evaluation process.

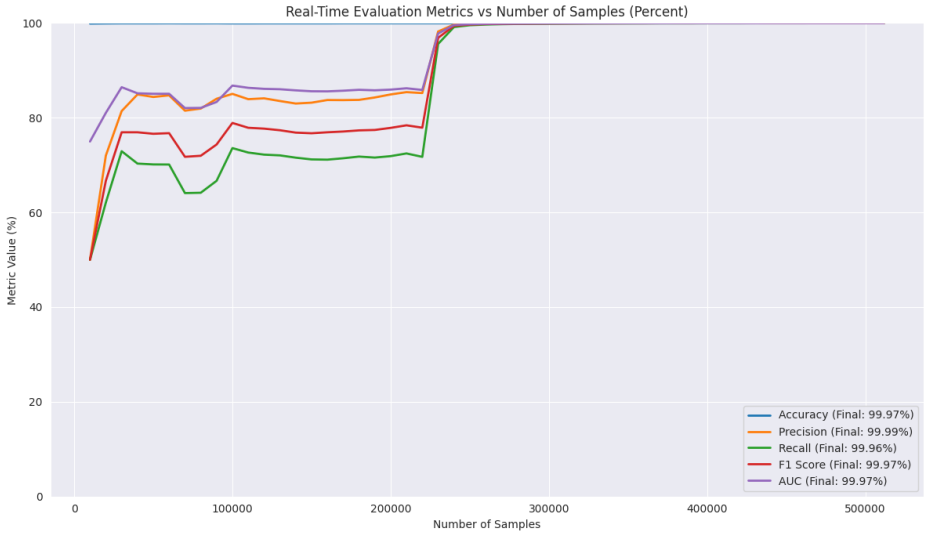


Fig 13. Real-Time Accuracy, Precision, Recall, F1 Score, and AUC progression of the model with ADWIN across the European Credit Cardholders Dataset (2013).

4.4 Model Performance on the European Credit Cardholders Dataset (2023)

The original dataset consisted of 30 features, including id, V1-V28, and Amount. To reduce dimensionality and eliminate redundancy, the MCS algorithm was applied with a correlation threshold of 0.6. The selected features include id, V4, V11, V12, V14, V18, V19, and V24, which collectively achieved a final classification accuracy of 0.9982. These selected attributes were introduced into the drift-based Online LSTM architecture, serving as the input space. The model reported performance at intervals of 10,000 evaluation steps, presenting results across the five selected metrics: Accuracy, Precision, Recall, F1-score, and AUC. Given that the proposed framework integrated the Online LSTM architecture with two distinct drift detection mechanisms, i.e., EDDM and ADWIN, results were collected and analysed for each configuration. The specific indices at which concept drift was detected were recorded, and at each detection point, the model, together with its optimizer, was reinitialised to enable effective adaptation to the new data distribution. Drifts were detected at indices 67173, 95156, 197481, and 227973 using the EDDM and at index 277151 using ADWIN. The Table 7 presents the results obtained at each evaluation step when the Online LSTM model was integrated with the Early Drift Detection Method (EDDM).

Table 7. Results of the Online LSTM integrated with EDDM on the European Credit Cardholders Dataset (2023).

Evaluation Step	Accuracy	Precision	Recall	F1-score	AUC
10,000	0.9987	0.4545	0.4167	0.4348	0.7080

20,000	0.9989	0.6250	0.5172	0.5660	0.7584
30,000	0.9991	0.7619	0.6667	0.7111	0.8332
40,000	0.9991	0.7778	0.6562	0.7119	0.8280
50,000	0.9992	0.7969	0.6623	0.7234	0.8310
60,000	0.9993	0.7867	0.6782	0.7284	0.8389
70,000	0.9992	0.7558	0.6311	0.6878	0.8154
80,000	0.9992	0.7386	0.6132	0.6701	0.8065
90,000	0.9992	0.7570	0.6429	0.6953	0.8213
100,000	0.9991	0.7976	0.6943	0.7424	0.8470
110,000	0.9991	0.7955	0.6965	0.7427	0.8481
120,000	0.9992	0.7966	0.6878	0.7382	0.8438
130,000	0.9992	0.7989	0.6777	0.7333	0.8387
140,000	0.9992	0.8010	0.6800	0.7356	0.8399
150,000	0.9992	0.8040	0.6780	0.7356	0.8389
160,000	0.9993	0.8107	0.6789	0.7389	0.8393
170,000	0.9993	0.8148	0.6795	0.7411	0.8397
180,000	0.9993	0.8246	0.6886	0.7505	0.8442
190,000	0.9993	0.8264	0.6849	0.7491	0.8424
200,000	0.9992	0.8008	0.6805	0.7358	0.8401
210,000	0.9993	0.8036	0.6842	0.7391	0.8420
220,000	0.9993	0.8014	0.6748	0.7327	0.8373
230,000	0.9992	0.9756	0.9515	0.9634	0.9756
240,000	0.9993	0.9954	0.9906	0.9930	0.9952
250,000	0.9993	0.9974	0.9948	0.9961	0.9973
260,000	0.9993	0.9982	0.9964	0.9973	0.9981
270,000	0.9994	0.9987	0.9973	0.9980	0.9985
280,000	0.9994	0.9989	0.9978	0.9983	0.9988
290,000	0.9994	0.9991	0.9981	0.9986	0.9989
300,000	0.9994	0.9992	0.9984	0.9988	0.9991
310,000	0.9994	0.9993	0.9986	0.9989	0.9992
320,000	0.9995	0.9994	0.9987	0.9991	0.9992
330,000	0.9995	0.9994	0.9989	0.9992	0.9993
340,000	0.9995	0.9995	0.9990	0.9992	0.9994
350,000	0.9995	0.9995	0.9991	0.9993	0.9994
360,000	0.9995	0.9996	0.9991	0.9993	0.9994
370,000	0.9995	0.9996	0.9992	0.9994	0.9995
380,000	0.9995	0.9996	0.9992	0.9994	0.9995
390,000	0.9996	0.9996	0.9993	0.9995	0.9995
400,000	0.9996	0.9997	0.9993	0.9995	0.9995
410,000	0.9996	0.9997	0.9994	0.9995	0.9996
420,000	0.9996	0.9997	0.9994	0.9996	0.9996
430,000	0.9996	0.9997	0.9994	0.9996	0.9996
440,000	0.9996	0.9997	0.9995	0.9996	0.9996
450,000	0.9996	0.9997	0.9995	0.9996	0.9996
460,000	0.9996	0.9998	0.9995	0.9996	0.9996

470,000	0.9996	0.9998	0.9995	0.9996	0.9996
480,000	0.9996	0.9998	0.9995	0.9997	0.9996
490,000	0.9996	0.9998	0.9996	0.9997	0.9997
500,000	0.9997	0.9998	0.9996	0.9997	0.9997
510,000	0.9997	0.9998	0.9996	0.9997	0.9997
511,767	0.9997	0.9998	0.9996	0.9997	0.9997

The graphs presented below illustrate the variation in Accuracy, Precision, Recall, F1-score, and AUC as successive samples are introduced into the model during the evaluation process.

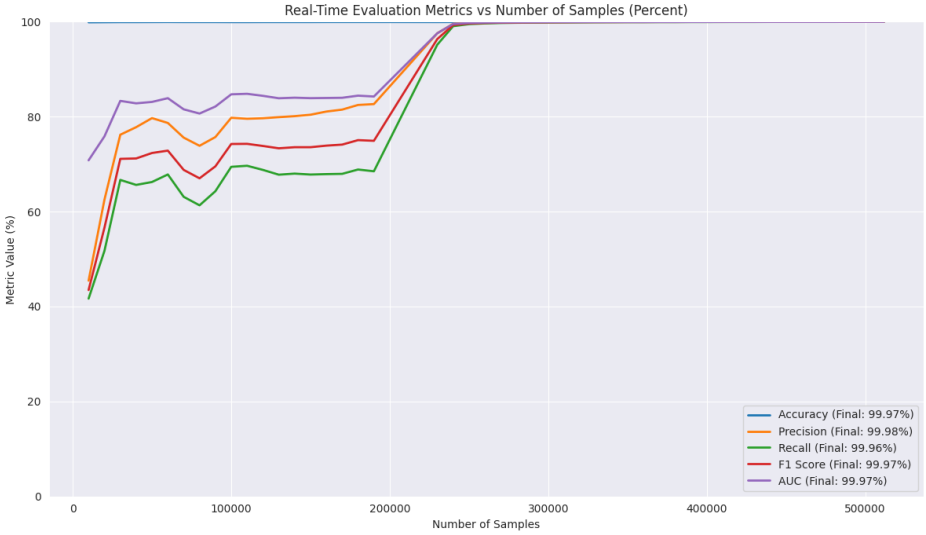


Fig 14. Real-Time Accuracy, Precision, Recall, F1 Score, and AUC progression of the model with EDDM across the European Credit Cardholders Dataset (2023).

The Table 8 presents the results obtained at each evaluation step when the Online LSTM model was integrated with Adaptive Windowing (ADWIN).

Table 8. Results of the Online LSTM integrated with ADWIN on the European Credit Cardholders Dataset (2023).

Evaluation Step	**Accuracy**	**Precision**	**Recall**	**F1-score**	**AUC**
10,000	0.9985	0.3846	0.4167	0.4000	0.7079
20,000	0.9987	0.5385	0.4828	0.5091	0.7411
30,000	0.9990	0.6889	0.6458	0.6667	0.8227
40,000	0.9990	0.7321	0.6406	0.6833	0.8201
50,000	0.9991	0.7576	0.6494	0.6993	0.8245
60,000	0.9992	0.7532	0.6667	0.7073	0.8332

70,000	0.9991	0.7303	0.6311	0.6771	0.8154
80,000	0.9992	0.7363	0.6321	0.6802	0.8159
90,000	0.9992	0.7685	0.6587	0.7094	0.8292
100,000	0.9991	0.8011	0.7306	0.7642	0.8651
110,000	0.9992	0.7946	0.7313	0.7617	0.8655
120,000	0.9992	0.7979	0.7317	0.7634	0.8657
130,000	0.9993	0.8010	0.7251	0.7612	0.8624
140,000	0.9993	0.8020	0.7200	0.7588	0.8599
150,000	0.9993	0.8048	0.7161	0.7578	0.8579
160,000	0.9993	0.8111	0.7154	0.7603	0.8576
170,000	0.9993	0.8114	0.7143	0.7598	0.8570
180,000	0.9993	0.8174	0.7216	0.7665	0.8607
190,000	0.9993	0.8196	0.7158	0.7642	0.8578
200,000	0.9993	0.8259	0.7125	0.7650	0.8561
210,000	0.9993	0.8309	0.7152	0.7687	0.8575
220,000	0.9993	0.8256	0.7052	0.7607	0.8525
230,000	0.9993	0.9786	0.9561	0.9672	0.9779
240,000	0.9994	0.9959	0.9915	0.9937	0.9957
250,000	0.9994	0.9978	0.9953	0.9965	0.9975
260,000	0.9994	0.9985	0.9968	0.9976	0.9983
270,000	0.9994	0.9988	0.9975	0.9982	0.9987
280,000	0.9994	0.9990	0.9978	0.9984	0.9988
290,000	0.9994	0.9992	0.9982	0.9987	0.9990
300,000	0.9995	0.9993	0.9984	0.9989	0.9991
310,000	0.9995	0.9994	0.9986	0.9990	0.9992
320,000	0.9995	0.9995	0.9988	0.9991	0.9993
330,000	0.9995	0.9995	0.9989	0.9992	0.9993
340,000	0.9995	0.9996	0.9990	0.9993	0.9994
350,000	0.9995	0.9996	0.9991	0.9993	0.9994
360,000	0.9995	0.9996	0.9991	0.9994	0.9995
370,000	0.9996	0.9996	0.9992	0.9994	0.9995
380,000	0.9996	0.9997	0.9993	0.9995	0.9995
390,000	0.9996	0.9997	0.9993	0.9995	0.9995
400,000	0.9996	0.9997	0.9993	0.9995	0.9996
410,000	0.9996	0.9997	0.9994	0.9996	0.9996
420,000	0.9996	0.9997	0.9994	0.9996	0.9996
430,000	0.9996	0.9998	0.9994	0.9996	0.9996
440,000	0.9996	0.9998	0.9995	0.9996	0.9996
450,000	0.9996	0.9998	0.9995	0.9996	0.9996
460,000	0.9996	0.9998	0.9995	0.9996	0.9996
470,000	0.9997	0.9998	0.9995	0.9997	0.9997
480,000	0.9997	0.9998	0.9996	0.9997	0.9997
490,000	0.9997	0.9998	0.9996	0.9997	0.9997
500,000	0.9997	0.9998	0.9996	0.9997	0.9997
510,000	0.9997	0.9998	0.9996	0.9997	0.9997

511,767	0.9997	0.9999	0.9996	0.9998	0.9997

The graphs presented below illustrate the variation in Accuracy, Precision, Recall, F1-score, and AUC as successive samples are introduced into the model during the evaluation process.

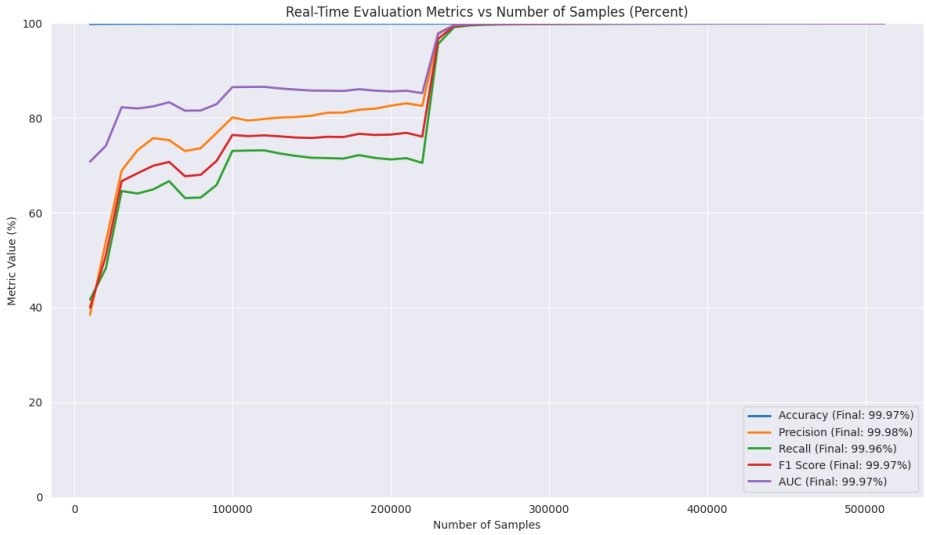


Fig 15. Real-Time Accuracy, Precision, Recall, F1 Score, and AUC progression of the model with ADWIN across the European Credit Cardholders Dataset (2023).

4.5 Model Performance on Credit Card Transactions Fraud Detection Dataset

The model was evaluated on a synthetically generated dataset produced using the Sparkov Data Generation Tool, designed to mimic real-world CCF patterns. The dataset originally comprised 23 features; however, the MCS algorithm, applied with a correlation threshold of 0.6, reduced this dimensionality to 5 selected features. Following three independent runs of the MCS algorithm, the final classification accuracy achieved was 0.8965. The selected features were amt, unix_time, hour, hour_sin, and hour_cos. The model reported performance at intervals of 100,000 evaluation steps, presenting results across the five selected metrics: Accuracy, Precision, Recall, F1-score, and AUC. Given that the proposed framework integrated the Online LSTM architecture with two distinct drift detection mechanisms, i.e., EDDM and ADWIN, results were collected and analysed for each configuration. The specific indices at which concept drift was detected were recorded, and at each detection point, the model, together with its optimiser, was reinitialised to enable effective adaptation to the new data distribution. The Table 9 below presents the results obtained at each evaluation step when the Online LSTM model was integrated with the Early Drift Detection Method (EDDM).

Table 9. Results of the Online LSTM integrated with EDDM on the Credit Card Transactions Fraud Detection Dataset

Evaluation Step	Accuracy	Precision	Recall	F1-score	AUC
100,000	0.9947	0.0364	0.0041	0.0074	0.5018
200,000	0.9953	0.0357	0.0034	0.0063	0.5015
300,000	0.9953	0.0303	0.0031	0.0057	0.5014
400,000	0.9948	0.0317	0.0031	0.0057	0.5013
500,000	0.9946	0.0280	0.0024	0.0045	0.5010
600,000	0.9948	0.0364	0.0031	0.0057	0.5013
700,000	0.9946	0.0317	0.0026	0.0048	0.5011
800,000	0.9945	0.0275	0.0022	0.0040	0.5009
900,000	0.9945	0.0246	0.0019	0.0036	0.5008
1,000,000	0.9943	0.0222	0.0017	0.0032	0.5007
1,100,000	0.9946	0.9934	0.9168	0.9536	0.9582
1,200,000	0.9950	0.9975	0.9667	0.9819	0.9832
1,300,000	0.9954	0.9984	0.9792	0.9887	0.9894
1,400,000	0.9957	0.9989	0.9849	0.9918	0.9922
1,500,000	0.9960	0.9991	0.9881	0.9936	0.9939
1,600,000	0.9963	0.9993	0.9902	0.9947	0.9949
1,700,000	0.9965	0.9994	0.9917	0.9955	0.9956
1,800,000	0.9967	0.9995	0.9928	0.9961	0.9962
1,900,000	0.9969	0.9995	0.9936	0.9966	0.9966
20,00,000	0.9970	0.9996	0.9943	0.9969	0.9969
2,100,000	0.9972	0.9996	0.9948	0.9972	0.9972
2,200,000	0.9973	0.9996	0.9952	0.9974	0.9974
2,300,000	0.9974	0.9997	0.9956	0.9976	0.9976
2,320,505	0.9974	0.9997	0.9957	0.9977	0.9976

The graphs presented below illustrate the variation in Accuracy, Precision, Recall, F1-score, and AUC as successive samples are introduced into the model during the evaluation process.

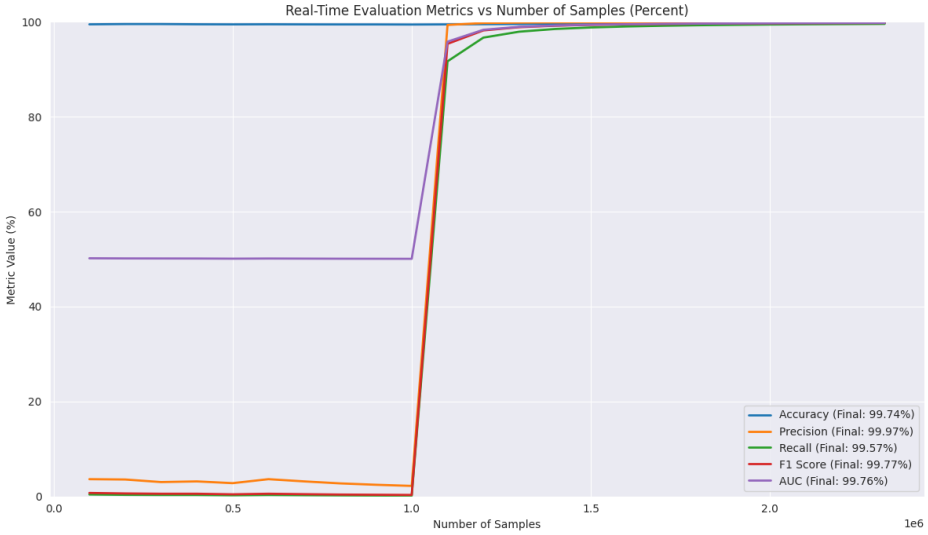


Fig 16. Real-Time Accuracy, Precision, Recall, F1 Score, and AUC progression of the model with EDDM across the Credit Card Transaction Fraud Detection Dataset.

The Table 10 presents the results obtained at each evaluation step when the Online LSTM model was integrated with Adaptive Windowing (ADWIN) on the Credit Card Transaction Fraud Detection Dataset.

Table 10. Results of the Online LSTM integrated with ADWIN on the Credit Crad Transaction Fraud Detection Dataset.

Evaluation Step	Accuracy	Precision	Recall	F1-score	AUC
100,000	0.9949	0.0370	0.0021	0.0039	0.5009
200,000	0.9955	0.0256	0.0011	0.0022	0.5005
300,000	0.9956	0.0182	0.0008	0.0015	0.5003
400,000	0.9951	0.0154	0.0005	0.0010	0.5002
500,000	0.9949	0.0211	0.0008	0.0016	0.5003
600,000	0.9950	0.0357	0.0014	0.0026	0.5006
700,000	0.9949	0.0323	0.0011	0.0022	0.5005
800,000	0.9947	0.0288	0.0010	0.0019	0.5004
900,000	0.9947	0.0355	0.0013	0.0025	0.5006
1,000,000	0.9946	0.0316	0.0011	0.0022	0.5005
1,100,000	0.9948	0.9969	0.9167	0.9551	0.9583
1,200,000	0.9952	0.9988	0.9667	0.9825	0.9832
1,300,000	0.9956	0.9993	0.9792	0.9891	0.9895
1,400,000	0.9959	0.9995	0.9849	0.9921	0.9923
1,500,000	0.9962	0.9996	0.9881	0.9938	0.9940
1,600,000	0.9964	0.9997	0.9902	0.9949	0.9950

1,700,000	0.9966	0.9997	0.9917	0.9957	0.9957
1,800,000	0.9968	0.9998	0.9928	0.9962	0.9963
1,900,000	0.9970	0.9998	0.9936	0.9967	0.9967
20,00,000	0.9971	0.9998	0.9943	0.9970	0.9970
2,100,000	0.9973	0.9998	0.9948	0.9973	0.9973
2,200,000	0.9974	0.9998	0.9952	0.9975	0.9975
2,300,000	0.9975	0.9998	0.9956	0.9977	0.9977
2,320,505	0.9975	0.9999	0.9957	0.9978	0.9978

The graphs presented below illustrate the variation in Accuracy, Precision, Recall, F1-score, and AUC as successive samples are introduced into the model during the evaluation process.

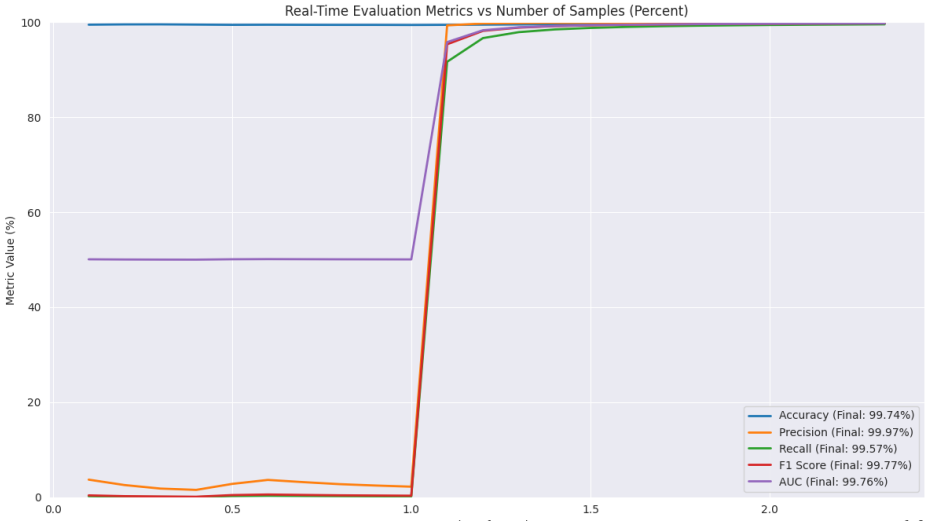


Fig 17. Real-Time Accuracy, Precision, Recall, F1 Score, and AUC progression of the model with ADWIN across the Credit Card Transaction Fraud Detection Dataset.

The Table 11 and 12 shows the final performance of the model across the three datasets, i.e., European Credit Cardholders Dataset (2013), European Credit Cardholders Dataset (2023), and the Credit Card Transaction Fraud Detection Dataset.

Table 11. Performance across the three datasets using Early Drift Detection Method (EDDM).

Datasets	**Accuracy**	**Precision**	**Recall**	**F1-score**	**AUC**
European Credit Cardholders Dataset (2013)	99.96%	99.98%	99.96%	99.97%	99.96%
European Credit Cardholders Dataset (2023)	99.97%	99.98%	99.96%	99.97%	99.97%

Credit Card Transaction Fraud Detection Dataset.	99.74%	99.97%	99.57%	99.77%	99.76%

Table 12. Performance across the three datasets using Adaptive Windowing (ADWIN).

Datasets	Accuracy	Precision	Recall	F1-score	AUC
European Credit Cardholders Dataset (2013)	99.97%	99.99%	99.96%	99.97%	99.97%
European Credit Cardholders Dataset (2023)	99.97%	99.98%	99.96%	99.97%	99.97%
Credit Card Transaction Fraud Detection Dataset.	99.75%	99.99%	99.57%	99.78%	99.78%

4.6 Performance Analysis on the Three Datasets Used

The evaluation of the proposed drift-based framework was learned in an online manner across with the LSTM updated as new samples arrived. The EDDM and ADWIN monitored and triggered resets when distributional shifts emerged. In the initial learning phase, when data arrived in batches, the model quickly showed a strong accuracy across all the three datasets. This is because the balanced class distribution ensured that both legitimate and fraudulent transaction patterns were equally represented from the start, allowing the model to make correct classifications even with limited data instances. However, precision, recall, F1-score, and AUC initially lagged behind accuracy. Unlike accuracy, which reflects the overall proportion of correct classification, these metrics capture fine-grained trade-offs in identifying fraudulent and legitimate transactions. In the early phase of the online learning, the model had not yet encountered enough diversity in the transaction behaviour to form effective decision boundaries. As more batches were processed, the model gained richer exposure to subtle variations in both legitimate and fraudulent behaviour. The drift detectors (EDDM and ADWIN) also enhanced this process by identifying distributional changes and triggering model resets, allowing the framework to recalibrate its international representations. This adaptation caused an improvement in sensitivity and specificity over time. This progressive learning explains the gradual increase in precision, recall, F1-score, and AUC toward their near-optimal levels, while accuracy remained consistently high from the beginning till the end.

4.7 Performance Comparison with Other Scholarly Works

While the proposed approach in this study demonstrates superior performance, it is important to compare it to existing works in the domain. To achieve this, the study is

compared against recent works the use either drift or non-drift detection approaches for CCF detection. Given the scarce and sensitive nature of datasets for CCF, the comparison will be limited to studies that utilise benchmark datasets frequently cited in literature. This focused selection ensures a fair basis for comparison.

Table 13 below provides a comparative analysis of works that employed a drift-based approach for CCF detection.

Table 13. Comparative Analysis with Recent Works

Author	Method	Dataset	Evaluation Metrics
Jabeen, et al., [46]	CNN-LSTM (CLST)	European Credit Cardholders Dataset (2013)	Precision: 83%, Recall: 99%, F1-score: 91%, ROC-AUC: 99.95%
Akour, Mohamed, & Salloum [47]	Hybrid CNN-LSTM-Attention	European Credit Cardholders Dataset (2013)	Accuracy: 99.93%, Precision: 90.51%, Recall: 89.27%, F1-score: 89.88%, AUC: 98%
Forough & Momtazi [48]	1. LSTM-CRF	European Credit Cardholders Dataset (2013)	Precision: 88.17%, Recall: 75.69%, F1-score: 80.76%, AUC-ROC: 87.83%, AUC-PR: 66.23%
	2. LSTM-LSTM-CRF		Precision: 83.55%, Recall: 75.25%, F1-score: 78.74%, AUC-ROC: 7.61%, AUC-PR: 62.58%
	1. LSTM-CRF	Brazilian Dataset	Precision: 92.56%, Recall: 79.55%, F1-score: 85.55%, AUC-ROC: 89.65%, AUC-PR: 74.39%
	2. LSTM-LSTM-CRF		Precision: 94.25%, Recall: 75.96%, F1-score: 83.65%, AUC-

			ROC: 87.89%, AUC-PR: 72.32%
Btoush E., Zhou, Gururajan, Chan, & Alsodi [49]	Hybrid stacking ML + DL (DTC, RF, SVM, XGBoost, CatBoost, LR, CNN, BiLSTM)	European Credit Cardholders Dataset (2013)	Accuracy: 99.97%, Precision: 97.62%, Recall: 83.67%, F1-score: 90.11, AUC: 91.83%
Larim, Kamal, Rahman, & Hossain [50]	RF + Gradient Boosting Machine + CatBoost	European Credit Cardholders Dataset (2013)	Accuracy: 98.11%, Precision: 96.60%, Recall: 99.74%, F1-score 98.14%
Proposed Model	Online LSTM with EDDM	European Credit Cardholders Dataset (2013)	Accuracy: 99.96%, Precision: 99.98%, Recall: 99.96%, F1-score: 99.97%, AUC: 99.96%
	Online LSTM with ADWIN		Accuracy: 99.97%, Precision: 99.99%, Recall: 99.96%, F1-score: 99.97%, AUC: 99.97%
Proposed Model	Online LSTM with EDDM	European Credit Cardholders Dataset (2023)	Accuracy: 99.97%, Precision: 99.98%, Recall: 99.96%, F1-score: 99.97%, AUC: 99.97%
	Online LSTM with ADWIN		Accuracy: 99.97%, Precision: 99.98%, Recall: 99.96%, F1-score: 99.97%, AUC: 99.97%
Proposed Model	Online LSTM with EDDM	Credit Card Transaction Fraud Detection Dataset (Sparkov Generated)	Accuracy: 99.74%, Precision: 99.97%, Recall: 99.57%, F1-score: 99.77%, AUC: 99.76%

Online LSTM with ADWIN	Accuracy: 99.75%, Precision: 99.99%, Recall: 99.57%, F1-score: 99.78%, AUC: 99.78%

4.8 Discussion

The proposed Online LSTM architecture integrated with drift detection mechanisms showed significant improvements in fraud detection performance across the three datasets. This enhanced performance can be attributed to the model's capacity to reset and retrain itself whenever a drift is detected considering the data came through in a streaming manner. This helped maintain adaptability to evolving patterns. To further enhance predictive accuracy, the study used the MCS algorithm for feature selection, effectively addressing the high dimensionality in the CCF dataset. Owing to the stochastic nature of MCS, the algorithm was run five times for each dataset, with a correlation threshold defined according to the characteristics of the dataset under consideration. Features that consistently emerged across five runs were ultimately selected, ensuring that the chosen features were both robust and relevant. This approach improved the robustness of the framework and contributed to superior performance. Furthermore, when compared to existing studies, it becomes evident that although many of earlier approaches achieve relatively good performance, they suffer from high false positives and negatives. This limitation is because such models are static in nature, making them incapable to cope with the dynamic and evolving characteristics of fraud patterns. Fraudulent behaviours adapt quickly, and as statistical properties of transaction shift over time, making static models lose their effectiveness. Again, almost all of the existing literature relied on a single dataset for experimentation and validation i.e. the European Credit Cardholders Dataset (2013.) While such an approach may simplify model development and evaluation, it restricts the generalizability of the finding.

5 Conclusion and Future Works

This study presents a drift-aware credit card fraud detection framework addressing concept drift, class imbalance, and high-dimensionality. The Online LSTM architecture adapts to evolving fraudulent behaviors, while EDDM and ADWIN detect gradual and abrupt drifts in transaction data. The Memory-Based Cuckoo Search (MCS) algorithm selects relevant features, and MAHAKIL, a biologically inspired oversampling method, generates diverse synthetic fraud samples. Experiments on three datasets—the European Credit Cardholders (2013 and 2023) and a synthetically generated fraud dataset—demonstrate high effectiveness, with Accuracy, Precision, Recall, F1-score, and AUC exceeding 99%, confirming robustness and generalizability. Future work includes integrating Explainable AI techniques to improve transparency and distinguish genuine drifts from noise.

Data Availability
The data used for this study is publicly available on Kaggle. The dataset can be found here: [https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud, https://www.kaggle.com/datasets/kartik2112/fraud-detection, https://www.kaggle.com/datasets/nelgiriyewithana/credit-card-fraud-detection-dataset-2023]

Conflict of Interest
The authors declare that there is no conflict of interest known to the authors

Funding
This study received no funding

Authors Contribution
AAD: Conceptualization, Methodology, Formal analysis
JKA: Conceptualization, Formal analysis, Investigation
DKA: Supervision, Software, Writing – original draft
LMB: Supervision, Visualization, Writing – review & editing
JYL: Resources, Software,
MATS: Resources, Validation,

References

1. Rahman, M., Ismail, I., Bahri, S., & Rahman, M. K. (2022, December 6). An Empirical Analysis of Cashless Payment Systems for Business Transactions. J. Open Innov. Technol. Mark. Complex, 8(4). doi: https://doi.org/10.3390/joitmc8040213.
2. Laxman, V., Ramesh, N., Prakash, S. K., & Aluvala, R. (2024, December). Emerging threats in digital payment and financial crime: A bibliometric review. Journal of Digital Economy, 3, 205-222. doi: https://doi.org/10.1016/j.jdec.2025.04.002.
3. Xu, L., Cui, L., Weise, T., Li, X., Wu, Z., Nie, F., . . . Tang, Y. (2021, December). Semi-supervised multi-Layer convolution kernel learning in credit evaluation. Pattern Recognition, Volume 120. doi: https://doi.org/10.1016/j.patcog.2021.108125.
4. Dornadula, V. N., & Geetha, S. (2019). Credit Card Fraud Detection using Machine Learning Algorithms. Procedia Computer Science, Volume 165, Pages 631-641. doi: https://doi.org/10.1016/j.procs.2020.01.057.
5. Zioviris, G., Kolomvatsos, K., & Stamoulis, G. (2024, March 27). An intelligent sequential fraud detection model based on deep learning. The Journal of Supercomputing, 80, 14824-14847. doi: https://doi.org/10.1007/s11227-024-06030-y.
6. Chatterjee, P., Das, D., & Rawat, D. B. (2024, September). Digital twin for credit card fraud detection: opportunities, challenges, and fraud detection advancements. Future Generation Computer Systems, Volume 158, Pages 410-426. doi: https://doi.org/10.1016/j.future.2024.04.057.
7. Makki, S., Assaghir, Z., Taher, Y., Haque, R., Hacid, M.-S., Zeineddine, & Hassan. (2019, July 8). An Experimental Study with Imbalanced Classification Approaches for Credit Card

Fraud Detection. IEEE Access, 93010 - 93022. doi: https://doi.org/10.1109/ACCESS.2019.2927266.

8. Strelcenia, E., & Prakoonwit, S. (2023, March 11). A Survey on GAN Techniques for Data Augmentation to Address the Imbalanced Data Issues in Credit Card Fraud Detection. Machine Learning & Knowledge Extraction, 304-329. doi: doi.org/10.3390/make5010019.
9. Bennin, K. E., Keung, J., Phannachitta, P., Monden, A., & Mensah, S. (2018, June 1). MAHAKIL: Diversity Based Oversampling Approach to Alleviate the Class Imbalance Issue in Software Defect Prediction. IEEE Transactions on Software Engineering, 44, 534 - 550. doi: https://doi.org/10.1109/TSE.2017.2731766.
10. Shahraki, A., Abbasi, M., Taherkordi, A., & Jurcut, A. D. (2022, February 16). A comparative study on online machine learning techniques for network traffic streams analysis. Computer Networks.
11. Wang, S., Minku, L. L., & Yao, X. (2018, January 4). A Systematic Study of Online Class Imbalance Learning with Concept Drift. IEEE Transactions on Neural Networks and Learning Systems, 29(10), 4802-4821. doi:10.1109/TNNLS.2017.2771290.
12. Agrahari, S., & Singh, A. K. (2022, November). Concept Drift Detection in Data Stream Mining: A literature review. Journal of King Saud University - Computer and Information Sciences, 34(10), 9523-9540. doi: https://doi.org/10.1016/j.jksuci.2021.11.006.
13. Ang, J. C., Mirzal, A., Haro, H., & Hamed, H. N. (2015, September 14). Supervised, Unsupervised, and Semi-Supervised Feature Selection: A Review on Gene Selection. IEEE/ACM Transactions on Computational Biology and Bioinformatics, 971-989. doi: https://doi.org/10.1109/TCBB.2015.2478454.
14. Wang, J., Zhang, Z., & Wang, Y. (2025, January 8). Utilizing Feature Selection Techniques for AI-Driven Tumor Subtype Classification: Enhancing Precision in Cancer Diagnostics. Biomolecules, 15(1). doi: https://doi.org/10.3390/biom15010081.
15. Gunantara, N., & Putra, D. N. (2019, January 6). The Characteristics of Metaheuristic Method in Selection of Path Pairs on Multicriteria Ad Hoc Networks. Journal of Computer Networks and Communications. doi: https://doi.org/10.1155/2019/7983583.
16. Trojovsky, P., & Dehghani, M. (2022, January 21). Pelican Optimization Algorithm: A Novel Nature-Inspired Algorithm for Engineering Applications. Sensors, 22(3). doi: https://doi.org/10.3390/s22030855.
17. Srivastava, A., Kundu, A., Shamik, S., & Arun K., M. (2008). Credit card fraud detection using Hidden Markov Model. IEEE Transactions on Dependable and Secure Computing, Pages 37 - 48.
18. Lucas, Y., Portier, P.-E., Laporte, L., He-Guelton, L., Olivier, C., Granitzer, M., & Clabretto, S. (2020, January). Towards automated feature engineering for credit card fraud detection using multi-perspective HMMs. Future Generation Computer Systems, Volume 102, Pages 393-402. doi: https://doi.org/10.1016/j.future.2019.08.029
19. Gelivi, S. C., Katragadda, D., Gurram, S. P., Rajkumar, .., & Jabbar, M. (2024). Credit Card Fraud Detection using Hidden Naive Bayes and Bayesian Belief Network. IEEE 9th International Conference for Convergence in Technology (I2CT).
20. Qawqzeh, Y. K., & Ashraf, M. (2023). A Fraud Detection System Using Decision Trees Classification in An Online Transactions. 12th International Conference on Software and Computer Applications. doi: https://doi.org/10.1145/3587828.3587860
21. Afriyie, J. K., Tawiah, K., Pels, W. A., Addai-Henne, S., Dwamena, H. A., Owiredu, E. O., . . . Eshun, J. (2023, January). A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions. Decision Analytics Journal. doi: https://doi.org/10.1016/j.dajour.2023.100163.

22. Karunya, R., Samyuktha, G., Muralidharan, D., Brindha, G., & Muthu, T. (2025, May 12). Credit Card Fraud Data Analysis and Prediction Using Machine Learning Algorithms. Security and Privacy. doi: https://doi.org/10.1002/spy2.70043.
23. Singhai, A., Aanjankumar, S., & Poonkuntran, S. (2023). A Novel Methodology for Credit Card Fraud Detection using KNN Dependent Machine Learning Methodology. Proceedings of the Second International Conference on Applied Artificial Intelligence and Computing (ICAAIC 2023).
24. Asha, R., & Suresh, K. K. (2021, January 23). Credit card fraud detection using artificial neural network. Global Transitions Proceedings, 35-41.
25. Rahman, S., & Yao, J. (2025). Credit Card Fraud Detection with Deep Multilayer Perceptrons. Machine Learning and Soft Computing, 146-158. doi: https://doi.org/10.1007/978-981-96-6403-0_13.
26. Gambo, M. L., Zainal, A., & Kassim, M. N. (2022, August). A Convolutional Neural Network Model for Credit Card Fraud Detection. International Conference on Data Science and Its Applications (ICoDSA). doi: https://doi.org/10.1109/ICoDSA55874.2022.9862930.
27. Potluri, P., Suneel, S., Chandra, B., Muthukumaran, D., Upadhyaya, M., & R, R. K. (2024). ELCCFD: An Efficient and Enhanced Credit Card Fraud Detection using Enhanced Deep Learning Principle. 5th International Conference on Image Processing and Capsule Networks (ICIPCN). doi: https://doi.org/10.1109/ICIPCN63822.2024.00062.
28. Roseline, J. F., Naidu, G., Pandi, V. S., Alamelu, S., & Mageswari, N. (2022, May 26). Autonomous credit card fraud detection using machine learning approach. Computers and Electrical Engineering. doi: https://doi.org/10.1016/j.compeleceng.2022.108132.
29. Ampofo, K. A., Owusu, E., & Appati, J. K. (2023, April 4). Performance Evaluation of LSTM Optimizers for Long-Term Electricity Consumption Prediction. 2022 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC). doi: https://doi.org/10.1109/ASSIC55218.2022.10088353.
30. Ida, S. J., Balasubadra, K., R, S. R., & T, L. N. (2024, May 28). Enhancing Credit Card Fraud Detection through LSTM-Based Sequential Analysis with Early Stopping. 2024 2nd International Conference on Networking and Communications (ICNWC). doi: https://doi.org/10.1109/ICNWC60771.2024.10537550.
31. Ranganatha, H., & Mustafa, A. S. (2024, August 23). Enhancing fraud detection efficiency in mobile transactions through the integration of bidirectional 3d Quasi-Recurrent Neural Network and blockchain technologies. Expert Systems With Applications. doi: https://doi.org/10.1016/j.eswa.2024.125179.
32. Zhang, C. (2025, February 12). A credit card fraud detection system based on Credit-Attention-GRU network. 2024 8th International Conference on Electrical, Mechanical and Computer Engineering (ICEMCE). doi:10.1109/ICEMCE64157.2024.10862244.
33. Mienye, I. D., & Swart, T. G. (2024, October 2). A Hybrid Deep Learning Approach with Generative Adversarial Network for Credit Card Fraud Detection. Technologies. doi: https://doi.org/10.3390/technologies12100186.
34. Deng, T., Bi, S., & Xiao, J. (2025, May 9). Transformer-Based Financial Fraud Detection with Cloud-Optimized Real-Time Streaming. BDEIM '24: Proceedings of the 2024 5th International Conference on Big Data Economy and Information Management, 702-707. doi: https://doi.org/10.1145/3724154.3724271.
35. Shi, S., Luo, W., & Pau, G. (2025, April 26). An attention-based balanced variational autoencoder method for credit card fraud detection. Applied Soft Computing. doi: https://doi.org/10.1016/j.asoc.2025.113190.
36. Yajid, M. S., Bhosle, N., Sudhamsu, G., Khatibi, A., Sharma, S., Jeet, R., . . . Santhosh, A. J. (2025, July 4). Hybrid Big Bang-Big crunch with cuckoo search for feature selection in

credit card fraud detection. Scientific Reports. doi: https://doi.org/10.1038/s41598-025-97149-2.

37. Ganji, V. R., Chaparala, A., & Sajja, R. (2023, January 30). Shuffled shepherd political optimization-based deep learning method for credit card fraud detection. Concurrency and Computation: Practice and Experience, 35(10). doi: https://doi.org/10.1002/cpe.7666.
38. Atimbire, S. A., Appati, J. K., & Owusu, E. (2024, February 24). Empirical exploration of whale optimization algorithm for heart disease prediction. Scientific Reports. doi: https://doi.org/10.1038/s41598-024-54990-1.
39. Ansah, P. A., Appati, J. K., Owusu, E., Boahen, E. K., Boakye-Sekyerehene, P., & Dwumfour, A. (2025, February 17). SB-YOLO-V8: A Multilayered Deep Learning Approach for Real-Time Human Detection. Engineering Reports. doi: https://doi.org/10.1002/eng2.70033.
40. Zhang, S.-s., Liu, J.-w., & Zuo, X. (2021, July). Adaptive online incremental learning for evolving data streams. Applied Soft Computing, 105. doi: https://doi.org/10.1016/j.asoc.2021.107255.
41. Ansah, P. A., Appati, J. K., Owusu, E., Boahen, E. K., Boakye-Sekyerehene, P., & Dwumfour, A. (2025, February 17). SB-YOLO-V8: A Multilayered Deep Learning Approach for Real-Time Human Detection. Engineering Reports. doi: https://doi.org/10.1002/eng2.70033.
42. Appati, J. K., Boante, L. M., Owusu, E., & Gah, S. K. (2023, September 16). Transfer Learning-Based Encoder-Decoder Model for Skin Lesion Segmentation. International Conference on Information, Communication and Computing Technology, 1841, 117-128. doi: https://doi.org/10.1007/978-3-031-43838-7_9.
43. Jain, S., Jain, A., & Jangid, M. (2023, April 25). Review of Metaheuristic Techniques for Feature Selection. Soft Computing: Theories and Applications, 397-410. doi:10.1007/978-981-19-9858-4_33.
44. Yang, X.-S., & Deb, S. (2009, December). Cuckoo Search Via Levy Flights. Proc. of World Congress in Nature & Biologically Inspired Computing (NaBIC 2009), 201-214.
45. Hinder, F., Vaquet, V., Brinkrolf, J., & Hammer, B. (2023, July 29). Model-based explanations of concept drift. Neurocomputing. doi: https://doi.org/10.1016/j.neucom.2023.126640.
46. Jabeen, M., Ramzan, S., Raza, A., Fitriyani, N. L., Syafrudin, M., & Lee, S. W. (2025, June 12). Enhanced Credit Card Fraud Detection Using Deep Hybrid CLST Model. Mathematics, 13(12). doi: https://doi.org/10.3390/math13121950.
47. Akour, I., Mohamed, N., & Salloum, S. (2025, June 25). Hybrid CNN-LSTM With Attention Mechanism for Robust Credit Card Fraud Detection. IEEE Access, 13, 114056-114068. doi: https://doi.org/10.1109/ACCESS.2025.3583253.
48. Forough, J., & Momtazi, S. (2021, August 2). Sequential credit card fraud detection: A joint deep neural network and probabilistic graphical model approach. Expert Systems. doi: https://doi.org/10.1111/exsy.12795.
49. Btoush, E., Zhou, X., Gururajan, R., Chan, K. C., & Alsodi, O. (2025, January 22). Achieving Excellence in Cyber Fraud Detection: A Hybrid ML+DL Ensemble Approach for Credit Cards. Applied Science, 15(3). doi: https://doi.org/10.3390/app15031081.
50. Larim, M. R., Kamal, M., Rahman, R. A., & Hossain, A. (2025, June 28). An Explainable Ensemble Model for Credit Card Fraud Detection Using Advanced Data Balancing and Feature Selection Technique. 2nd International Conference on Next-Generation Computing, IoT and Machine Learning (NCIM 2025).